

Unit 1

Database and Database Users

Introduction

- Databases and database technology have had a major impact on the growing use of computers
- Databases play a critical role in almost all areas where computers are used, including business, e-commerce, social media, engineering, medicine, genetics, law, education etc
- A database is a collection of related data
- Data means known facts that can be recorded and that have implicit meaning

A database has the following implicit properties:

- A database represents some aspect of the real world, sometimes called the miniworld. Changes to the miniworld are reflected in the database.
- A database is a logically coherent collection of data with some inherent meaning. A random assortment of data cannot correctly be referred to as a database.
- A database is designed, built, and populated with data for a specific purpose. It has an intended group of users and some preconceived applications in which these users are interested.

- A **database management system (DBMS)** is a computerized system that enables users to create and maintain a database.
- The DBMS is a general-purpose software system that facilitates the processes of defining, constructing, manipulating, and sharing databases among various users and applications.

Typical DBMS Functionality

- **Defining** a database involves specifying the data types, structures, and constraints of the data to be stored in the database. The database definition or descriptive information is also stored by the DBMS in the form of a database catalog or dictionary; it is called **meta-data**.
- **Constructing** the database is the process of storing the data on some storage medium that is controlled by the DBMS.
- **Manipulating** a database includes functions such as querying the database to retrieve specific data, updating the data base to reflect changes in the miniworld, and generating reports from the data.
- **Sharing** a database allows multiple users and programs to access the database simultaneously.

- **Protection** includes system protection against hardware or software malfunction (or crashes) and security protection against unauthorized or malicious access.
- A typical large database may have a life cycle of many years, so the DBMS must be able to **maintain** the database system by allowing the system to evolve as requirements change over time.

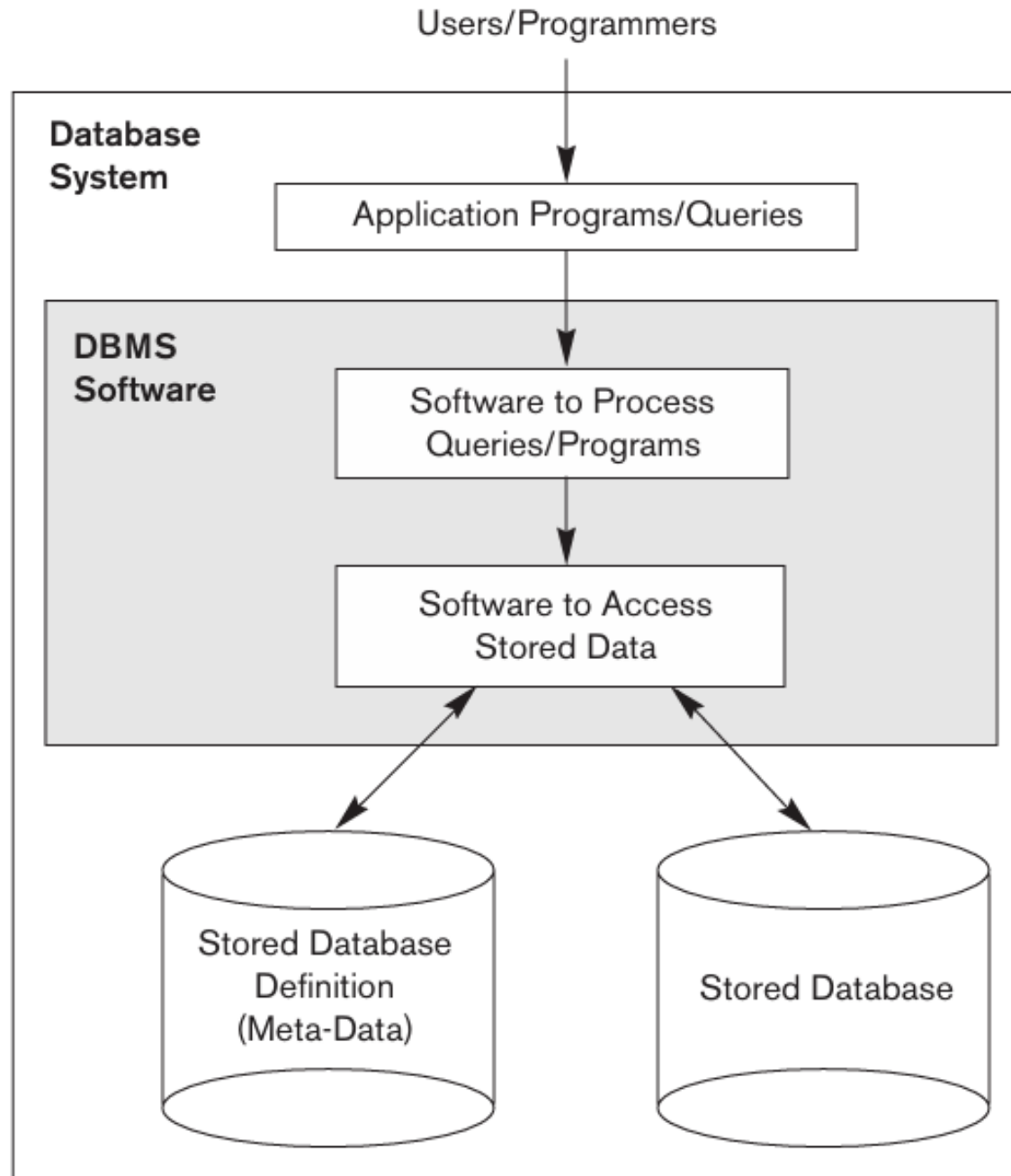


Figure 1.1
A simplified database
system environment.

An Example - UNIVERSITY database

A UNIVERSITY database for maintaining information concerning students, courses, and grades in a university environment.

STUDENT

Name	Student_number	Class	Major
Smith	17	1	CS
Brown	8	2	CS

COURSE

Course_name	Course_number	Credit_hours	Department
Intro to Computer Science	CS1310	4	CS
Data Structures	CS3320	4	CS
Discrete Mathematics	MATH2410	3	MATH
Database	CS3380	3	CS

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
85	MATH2410	Fall	07	King
92	CS1310	Fall	07	Anderson
102	CS3320	Spring	08	Knuth
112	MATH2410	Fall	08	Chang
119	CS1310	Fall	08	Anderson
135	CS3380	Fall	08	Stone

GRADE_REPORT

Student_number	Section_identifier	Grade
17	112	B
17	119	C
8	85	A
8	92	A
8	102	B
8	135	A

PREREQUISITE

Course_number	Prerequisite_number
CS3380	CS3320
CS3380	MATH2410
CS3320	CS1310

Some mini-world relationships:

STUDENTs take SECTIONs

COURSEs have prerequisite COURSEs

INSTRUCTORs teach SECTIONs

COURSEs are offered by DEPARTMENTs

STUDENTs major in DEPARTMENTs

- Design of a new application for an existing database or design of a brand-new database starts off with a phase called **requirements specification and analysis**.
- These requirements are documented in detail and transformed into a **conceptual design** that can be represented and manipulated using some computerized tools so that it can be easily maintained, modified, and transformed into a database implementation (**Entity-Relationship model**)
- The design is then translated to a **logical design** that can be expressed in a data model implemented in a commercial DBMS.
- The final stage is **physical design**, during which further specifications are provided for storing and accessing the database.
- The database design is implemented, populated with actual data, and continuously maintained to reflect the state of the miniworld.

Characteristics of the Database Approach

1. Self-Describing Nature of a Database System

- A fundamental characteristic of the database approach is that the database system contains not only the database itself but also a complete definition or description of the database structure and constraints.
- This definition is stored in the DBMS catalog, which contains information such as the structure of each file, the type and storage format of each data item, and various constraints on the data.
- The information stored in the catalog is called **meta-data**, and it describes the structure of the primary database

RELATIONS

Relation_name	No_of_columns
STUDENT	4
COURSE	4
SECTION	5
GRADE_REPORT	3
PREREQUISITE	2

COLUMNS

Column_name	Data_type	Belongs_to_relation
Name	Character (30)	STUDENT
Student_number	Character (4)	STUDENT
Class	Integer (1)	STUDENT
Major	Major_type	STUDENT
Course_name	Character (10)	COURSE
Course_number	XXXXNNNN	COURSE
....		
....		
....		
Prerequisite_number	XXXXNNNN	PREREQUISITE

Figure 1.3

An example of a database catalog for the database in Figure 1.2.

2. Insulation between Programs and Data and Data Abstraction

- The structure of data files is stored in the DBMS catalog separately from the access programs. We call this property **program-data independence**
- For example, if we want to add another piece of data to each STUDENT record, say the Birth_date, we only need to change the description of STUDENT records in the catalog to reflect the inclusion of the new data item Birth_date; no programs are changed. The next time a DBMS program refers to the catalog, the new structure of STUDENT records will be accessed and used.
- **Data Abstraction-** A DBMS provides users with a conceptual representation of data that does not include many of the details of how the data is stored or how the operations are implemented.

3. Support of Multiple Views of the Data

- A database typically has many types of users, each of whom may require a different perspective or view of the database.
- A view may be a subset of the database, or it may contain virtual data that is derived from the database files but is not explicitly stored.
- A multiuser DBMS whose users have a variety of distinct applications must provide facilities for defining multiple views.
- For example, one user of the database may be interested only in accessing and printing the transcript of each student; the view for this user is shown in Fig

TRANSCRIPT

Student_name	Student_transcript				
	Course_number	Grade	Semester	Year	Section_id
Smith	CS1310	C	Fall	08	119
	MATH2410	B	Fall	08	112
Brown	MATH2410	A	Fall	07	85
	CS1310	A	Fall	07	92
	CS3320	B	Spring	08	102
	CS3380	A	Fall	08	135

- A second user, who is interested only in checking that students have taken all the prerequisites of each course for which the student registers, may require the view shown in Fig

COURSE_PREREQUISITES

Course_name	Course_number	Prerequisites
Database	CS3380	CS3320
		MATH2410
Data Structures	CS3320	CS1310

4. Sharing of Data and Multiuser Transaction Processing

- A multiuser DBMS, as its name implies, must allow multiple users to access the database at the same time. This is essential if data for multiple applications is to be integrated and maintained in a single database.
- The DBMS must include concurrency control software to ensure that several users trying to update the same data do so in a controlled manner so that the result of the updates is correct.
- For example, when several reservation agents try to assign a seat on an airline flight, the DBMS should ensure that each seat can be accessed by only one agent at a time for assignment to a passenger. These types of applications are generally called **online transaction processing (OLTP) applications**.
- A fundamental role of multiuser DBMS software is to ensure that concurrent transactions operate correctly and efficiently.

A Brief History of Database Applications

Early Database Applications Using Hierarchical and Network Systems

- Early database systems were widely used in organizations like corporations, universities, hospitals, and banks to store large volumes of similar records such as student details, courses, and grades. These systems handled many types of records with complex relationships.
- However, a major limitation was the lack of **data abstraction and program–data independence**. Conceptual relationships were tightly linked with physical storage on disk. Although this design allowed fast access for specific queries, it made the system **inflexible** when new queries or requirements arose. Reorganizing data or changing storage structures was difficult and time-consuming.
- Another drawback was that these systems relied only on **programming language interfaces**, making the development of new queries costly and slow, as new programs had to be written and tested each time.

Providing Data Abstraction and Application Flexibility with Relational Databases

- Relational databases were introduced to separate how data is **physically stored** from how it is **logically represented**, using a strong mathematical foundation.
- They use **tables** and support **high-level query languages like SQL**, which made writing and changing queries much faster compared to programming-only systems.
- This improved **data abstraction** and **program–data independence**, allowing databases to be easily reorganized as requirements changed.
- Although early relational systems in the 1970s and early 1980s were slow, later improvements in **indexing, storage methods, and query optimization** greatly enhanced their performance.

Object-Oriented Applications and the Need for More Complex Databases

- Object-oriented databases (OODBs) were developed in the 1980s to support object-oriented programming and to store complex, structured data.
- They offered advanced features such as **encapsulation, inheritance, abstract data types, and object identity**, making them more powerful than traditional relational databases in some cases.
- Initially, OODBs were seen as competitors to relational databases.
- However, due to their **complexity** and the **lack of early standards**, their adoption remained limited.
- Today, they are mainly used in **specialized areas** like engineering design, multimedia applications, and manufacturing systems.
- Instead of fully replacing relational databases, many object-oriented features were later added to relational DBMSs, resulting in **object-relational DBMSs (ORDBMSs)**.

Interchanging Data on the Web for E-Commerce Using XML

- The World Wide Web connects computers across the globe and allows users to create **static web pages** using languages like **HTML**, which are stored on web servers and viewed through browsers.
- These pages are connected using **hyperlinks**.
- From the 1990s onward, **e-commerce** became a major web application, where important information such as product prices, availability, and flight details is **dynamically retrieved from databases (DBMSs)**.
- To support the exchange of such dynamic data between databases and web pages, several technologies were developed.
- **XML (eXtended Markup Language)** became a standard for data interchange, combining features of document systems with database modeling concepts.

Extending Database Capabilities for New Applications

- Database systems began to be used for **new types of applications** beyond traditional business data.
- These applications included **scientific research, image and video storage, data mining, spatial systems, and time-series data**.
- Earlier, such applications used their own special software and file structures, but later database systems tried to support them.
- However, **basic relational databases were not well suited** for these applications because
 - Needed **complex data structures** instead of simple tables
 - Required **new data types** (images, videos, spatial data, time values)
 - Needed **new operations and query features**
 - Required **special storage and indexing methods** for fast searching
- To overcome these limitations, DBMS developers added **new functionalities**. Some were **general-purpose**, such as object-oriented features added to relational databases, while others were **special-purpose modules**, like time-series or spatial modules, which users could add based on their application needs.

Emergence of Big Data Storage Systems and NOSQL Databases

- The growth of platforms such as **social media websites, e-commerce companies, web search engines, and cloud storage services** caused a massive increase in the amount of data stored in large databases.
- This data included **nontraditional data** like social media posts, tweets, and logs, which needed **fast access, scalability, and reliable storage**.
- Traditional **SQL relational databases** were not suitable for all these requirements.
- As a result, **new types of database systems called NoSQL databases** were developed.
- NoSQL (Not Only SQL) means that large data systems may use a **combination of SQL and NoSQL databases**, depending on the application's needs, to efficiently manage huge volumes of data.

Advantages of Using the DBMS Approach

Controlling Redundancy

- Redundancy means storing the same data in more than one place.
- For example, in a university system, a student's name and date of birth may be stored separately in both the registration and accounting files.
- If one file is updated incorrectly or not updated, the student information becomes inconsistent.
- A DBMS reduces redundancy by storing data in one place and controls it using integrity constraints.

Without Controlling Redundancy (Single Table)

STUDENT_INFO

StudentID	Name	Dept	HOD
101	Ramesh	CSE	Dr. Rao
102	Sita	CSE	Dr. Rao
103	Anil	ECE	Dr. Kumar

After Controlling Redundancy (Normalized Tables)

STUDENT Table

StudentID	Name	DeptID
101	Ramesh	D1
102	Sita	D1
103	Anil	D2

DEPARTMENT Table

DeptID	DeptName	HOD
D1	CSE	Dr. Rao
D2	ECE	Dr. Kumar

Restricting Unauthorized Access

- In a shared database, not all users are allowed to access all data.
- A DBMS controls **security and authorization** by giving users accounts with passwords and specific permissions.
- For example, in a company database, only HR staff can view or update employee salaries, while other employees can only view general information.
- The DBMS automatically enforces these access restrictions to protect confidential data.

Providing Persistent Storage for Program Objects

- Databases provide **persistent storage** for program objects and data structures, so they are not lost after a program stops.
- Object-oriented databases can store complex objects directly and automatically handle conversions between programs and the database.
- Traditional DBMS stores data in **tables (rows and columns)**. But programming languages use **objects and classes**.
- So, programmers had to **convert objects → tables → objects again**. This conversion problem is called **Impedance Mismatch Problem**.
- Object-oriented DBMS can store **objects directly**, without conversion. DBMS automatically converts program objects to database format.

Providing Storage Structures and Search Techniques for Efficient Query Processing

- A DBMS improves query and update performance by using **indexes** (such as tree or hash structures) to quickly locate records on disk and **buffering** to keep frequently used data in main memory.
- The **query processing and optimization module** selects the most efficient execution plan for each query.
- Deciding which indexes to create and maintain is part of **physical database design**, managed by the DBA.

Providing Backup and Recovery

- A DBMS must provide facilities for recovering from hardware or software failures.
- The backup and recovery subsystem of the DBMS is responsible for recovery.
- For example, if the computer system fails in the middle of a complex update transaction, the recovery subsystem is responsible for making sure that the database is restored to the state it was in before the transaction started executing.
- Disk backup is also necessary in case of a catastrophic disk failure.

Providing Multiple User Interfaces

- A DBMS must support **different user interfaces** because users have varying technical skills.
- It provides **mobile apps, query languages** for casual users, **programming interfaces** for developers, **forms and commands** for parametric users, and **menu-driven or natural language interfaces** for standalone users.
- Many of these are **graphical user interfaces (GUIs)**, and modern DBMSs also support **Web-based GUI access** to databases.

Representing Complex Relationships among Data

- A database contains different types of data that are connected through **multiple relationships**.
- For example, one student record can be linked to many grade records, and each section can be linked to one course and many students.
- A DBMS must support representing these **complex relationships**, allow new relationships to be defined, and enable efficient retrieval and updating of related data.

Enforcing Integrity Constraints

- A DBMS must support **defining and enforcing integrity constraints** to ensure data accuracy and correctness. The simplest constraints are **data type constraints**, such as specifying that *Class* must be an integer or *Name* must be a string of limited length. Additional constraints can restrict values to a specific range, such as *Class* between 1 and 5.
- More complex constraints include **referential integrity constraints**, which ensure that related records exist (e.g., every section must be linked to a valid course).
- **Uniqueness (key) constraints** ensure that certain values, like *Course_number*, are unique.
- Some constraints are enforced automatically by the DBMS, while others must be checked by programs or during data entry; these are commonly called **business rules**.

Permitting Inferencing and Actions Using Rules and Triggers

- Some database systems support **deduction rules** to infer new information automatically from stored data; such systems are called **deductive database systems**. For example, rules can determine whether a student is on probation without writing procedural code. If rules change, it is easier to modify the declared rules than to rewrite programs.
- Modern relational databases use **triggers**, which automatically execute actions when data is inserted, updated, or deleted.
- **Stored procedures** are predefined programs stored in the database to enforce complex rules.
- **Active database systems** extend this idea by automatically initiating actions when specific events and conditions occur, making databases more intelligent and responsive.

Additional Implications of Using the Database Approach

- **Potential for Enforcing Standards:** The DBA can define and enforce standards for data names, formats, reports, and terminology in a centralized database, improving communication and consistency across departments.
- **Reduced Application Development Time:** Once a database is established, new applications and reports can be developed quickly using DBMS tools, much faster than file-based systems.
- **Flexibility:** DBMS allows changes in database structure (like adding new data fields or tables) without affecting existing data and applications.
- **Availability of Up-to-Date Information:** All users can access the latest data immediately after updates, which is essential for real-time systems like banking and reservations.
- **Economies of Scale:** Centralized data and applications reduce duplication and allow organizations to invest in shared high-performance hardware, lowering overall costs.

Database System Concepts and Architecture

Data Models, Schemas and Instances

- A **data model** is a collection of concepts that can be used to **describe the structure of a database**.
- Structure of a database means the **data types, relationships and constraints that apply to the data**.
- Most data models also include a **set of basic operations for specifying retrievals and updates on the database**.

Data Model Structure and Constraints

- Constructs are used to define the database structure
- Constructs typically include **elements** (and their **data types**) as well as groups of elements (e.g. **entity, record, table**), and **relationships** among such groups
- Constraints specify some restrictions on valid data; these constraints must be enforced at all times

Data Model Operations

- These operations are used for specifying database retrievals and updates by referring to the constructs of the data model.
- Operations on the data model may include:
 - ✓ **basic model operations** (e.g. generic insert, delete, update) and
 - ✓ **user-defined operations** (e.g. compute_student_gpa, update_inventory)

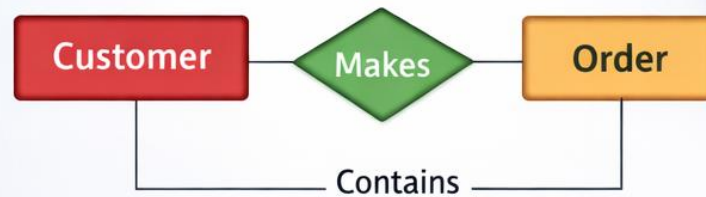
Categories of Data Models

- **High-level or conceptual data models** provide concepts that are close to the way many users perceive data
- **Low-level or physical data models** provide concepts that describe the details of how data is stored on the computer storage media. Concepts provided by physical data models are generally meant for computer specialists, not for end users.
- **Representational or implementation data models** lie between conceptual and physical data models. These models are easy for users to understand and close to how data is stored, hiding low-level storage details but allowing direct implementation.

Conceptual Data Model

— High-Level Design —

- Entities & Relationships
- Business Concepts
- No Technical Details

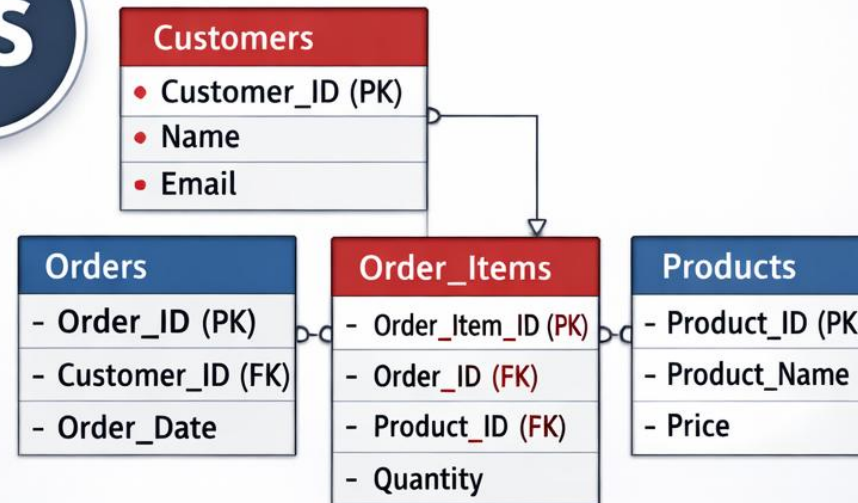


VS

Physical Data Model

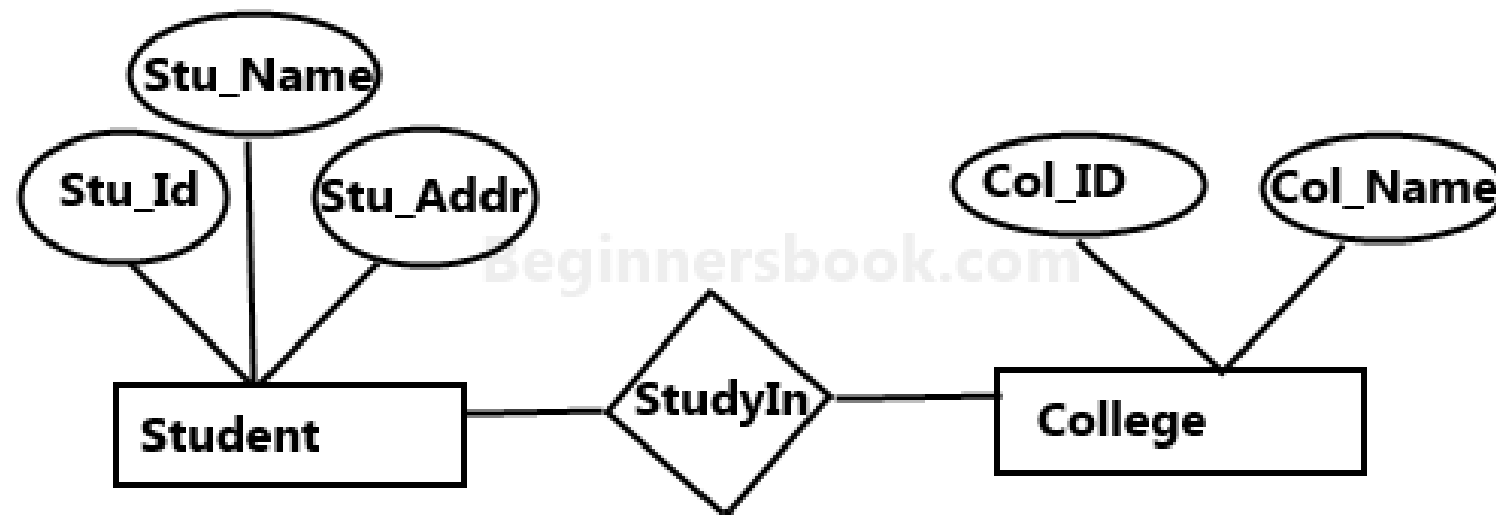
— Database Implementation —

- Tables & Columns
- Data Types & Keys
- Primary & Foreign Keys



Conceptual data models

- Conceptual data models use concepts such as entities, attributes and relationships.
- An **entity** represents a real-world object or concept, such as an employee or a project from the miniworld that is described in the database.
- An **attribute** represents some property of interest that further describes an entity, such as the employee's name or salary.
- A **relationship** among two or more entities represents an association among the entities, for example, a works-on relationship between an employee and a project.
- Ex: **entity–relationship model** is a popular high-level conceptual data model



Sample E-R Diagram

Representational or implementation data models

- Representational or implementation data models are the models used most frequently in traditional commercial DBMSs.
- These include the widely used **relational data model**
- Representational data models represent data by using record structures and hence are sometimes called record-based data models.

Physical data models

- Physical data models describe how data is stored as files in the computer by representing information such as record formats, record orderings, and access paths.
- An **access path** is a search structure that makes the search for particular database records efficient, such as indexing or hashing.
- An **index** is an example of an access path that allows direct access to data using an index term or a keyword.

Schemas, Instances and Database State

Database Schema

- The description of a database is called the **database schema**, which is specified during database design and is not expected to change frequently.
- It includes descriptions of the database structure, data types, and the constraints on the database.
- An illustrative display of database schema is called a **schema diagram**.
- The diagram displays the structure of each record type but not the actual instances of records.
- Each object in the schema—such as STUDENT or COURSE is called a **schema construct**.

STUDENT

Name	Student_number	Class	Major
------	----------------	-------	-------

COURSE

Course_name	Course_number	Credit_hours	Department
-------------	---------------	--------------	------------

PREREQUISITE

Course_number	Prerequisite_number
---------------	---------------------

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
--------------------	---------------	----------	------	------------

GRADE_REPORT

Student_number	Section_identifier	Grade
----------------	--------------------	-------

Fig: Schema diagram

Database State

- The data in the database at a particular moment in time is called a **database state or snapshot**.
- It is also called the current set of occurrences or instances in the database.
- Many database states can be constructed to correspond to a particular database schema.
- Every time we insert or delete a record or change the value of a data item in a record, we change one state of the database into another state.

COURSE

Course_name	Course_number	Credit_hours	Department
Intro to Computer Science	CS1310	4	CS
Data Structures	CS3320	4	CS
Discrete Mathematics	MATH2410	3	MATH
Database	CS3380	3	CS

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
85	MATH2410	Fall	04	King
92	CS1310	Fall	04	Anderson
102	CS3320	Spring	05	Knuth
112	MATH2410	Fall	05	Chang
119	CS1310	Fall	05	Anderson
135	CS3380	Fall	05	Stone

GRADE_REPORT

Student_number	Section_identifier	Grade
17	112	B
17	119	C
8	85	A
8	92	A
8	102	B
8	135	A

PREREQUISITE

Course_number	Prerequisite_number
CS3380	CS3320
CS3380	MATH2410
CS3320	CS1310

Fig: Database state

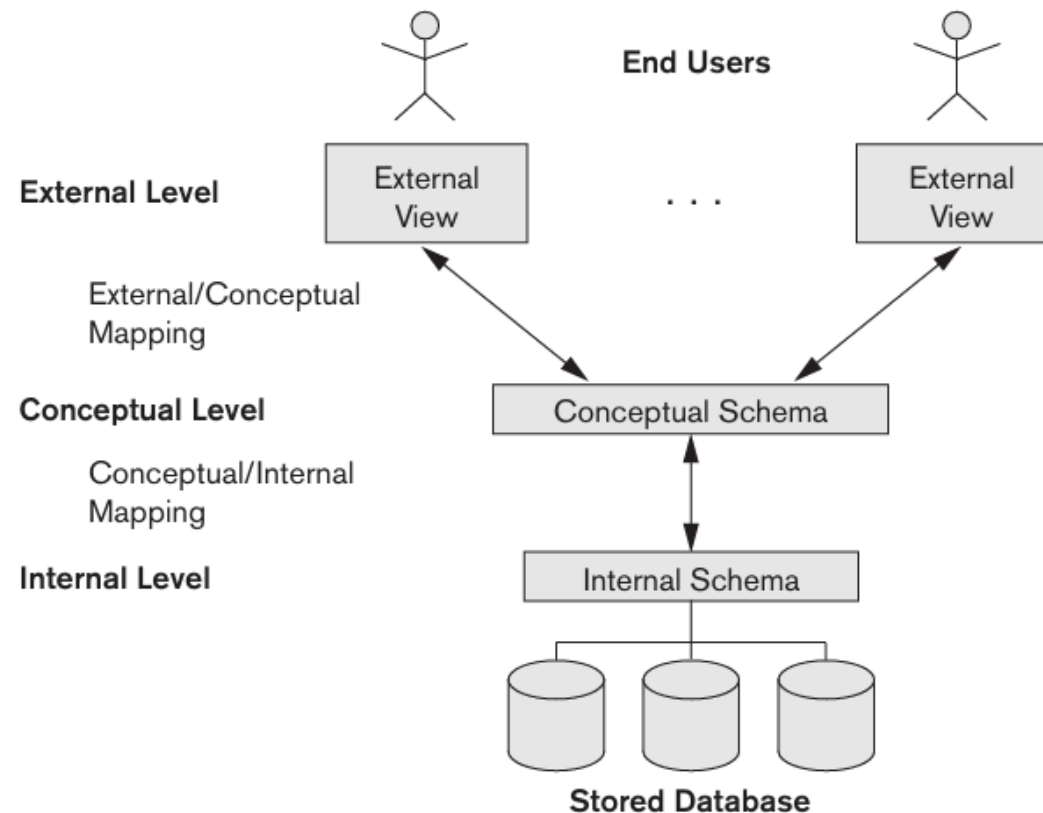
Database schema vs Database state

- When we define a new database, we specify its database schema only to the DBMS.
- At this point, the corresponding database state is the empty state with no data.
- We get the initial state of the database when the database is first populated or loaded with the initial data.
- From then on, every time an update operation is applied to the database, we get another database state.
- At any point in time, the database has a current state.
- The DBMS is partly responsible for ensuring that every state of the database is a **valid state**—that is, a state that satisfies the structure and constraints specified in the schema.

- The DBMS stores the descriptions of the schema constructs and constraints—also called the **meta-data**—in the DBMS **catalog** so that DBMS software can refer to the schema whenever it needs to.
- The **schema** is sometimes called the **intension**, and a **database state** is called an **extension of the schema**.
- Although, the schema is not supposed to change frequently, it is not uncommon that changes occasionally need to be applied to the schema as the application requirements change. This is known as **schema evolution**.
- For example, if another data item needs to be stored for each record in a file, such as adding the Date_of_birth to the STUDENT schema.

Three-Schema Architecture and Data Independence

The Three-Schema Architecture



The goal of the three-schema architecture is to separate the user applications from the physical database.

In this architecture, schemas can be defined at the following three levels:

1. The internal level has an **internal schema**, which describes the physical storage structure of the database. The internal schema uses a physical data model and describes the complete details of data storage and access paths for the database.
2. The conceptual level has a **conceptual schema**, which describes the structure of the whole database for a community of users. The conceptual schema hides the details of physical storage structures and concentrates on describing entities, data types, relationships, user operations, and constraints. Usually, a representational data model is used to describe the conceptual schema.

3. The external or view level includes a number of **external schemas or user views**. Each external schema describes the part of the database that a particular user group is interested in and hides the rest of the database from that user group. As in the previous level, each external schema is typically implemented using a representational data model.

Mappings among schema levels are needed to transform requests and data

- In the three-schema architecture, each user group refers to its own external schema.
- Hence, the DBMS must transform a request specified on an external schema into a request against the conceptual schema, and then into a request on the internal schema for processing over the stored database.
- If the request is a database retrieval, the data extracted from the stored database must be reformatted to match the user's external view.
- The processes of transforming requests and results between levels are called **mappings**.

Data Independence

The three-schema architecture support data independence, which can be defined as the capacity to change the schema at one level of a database system without having to change the schema at the next higher level.

Two types of data independence

1. Logical Data Independence
2. Physical Data Independence

Logical data independence

- Logical data independence is the capacity to change the conceptual schema without having to change external schemas or application programs.
- We may change the conceptual schema to expand the database (by adding a record type or data item), to change constraints, or to reduce the database (by removing a record type or data item).
- After the conceptual schema undergoes a logical reorganization, application programs that reference the external schema constructs must work as before.

Example of Logical Data Independence

Original Conceptual Schema

- A **Student** database has a table:

STUDENT(StudentID, Name, Dept)

- An external schema (view) used by an application shows:

StudentID, Name

Change in Conceptual Schema

- Later, the database designer **adds a new attribute**:

STUDENT(StudentID, Name, Dept, Email)

Logical Data Independence

- The **external schema (view)** still shows only StudentID and Name.
- The **application program does NOT need to be changed.**
- Only the **conceptual schema and mappings are updated.**

Physical data independence

- Physical data independence is the capacity to change the internal schema without having to change the conceptual schema. Hence, the external schemas need not be changed as well.
- Changes to the internal schema may be needed because some physical files were reorganized—for example, by creating additional access structures—to improve the performance of retrieval or update.
- If the same data as before remains in the database, we should not have to change the conceptual schema.

Example of Physical Data Independence

Original Physical Storage

- A **STUDENT** table is stored:
- In a **heap file** (unsorted storage)
- Without any index

Change in Physical Schema

The DBA decides to improve performance by:

- Adding an **index on StudentID**
- Changing storage from **heap file** → **B+ tree index file**
- Moving the database to a **faster SSD disk**

Physical Data Independence

- The **STUDENT** table structure does **NOT** change
- The **conceptual schema** remains same
- The **applications** continue working without any change
- Only the **internal (physical) schema** is modified.

The Database System Environment

1. DBMS Component Modules

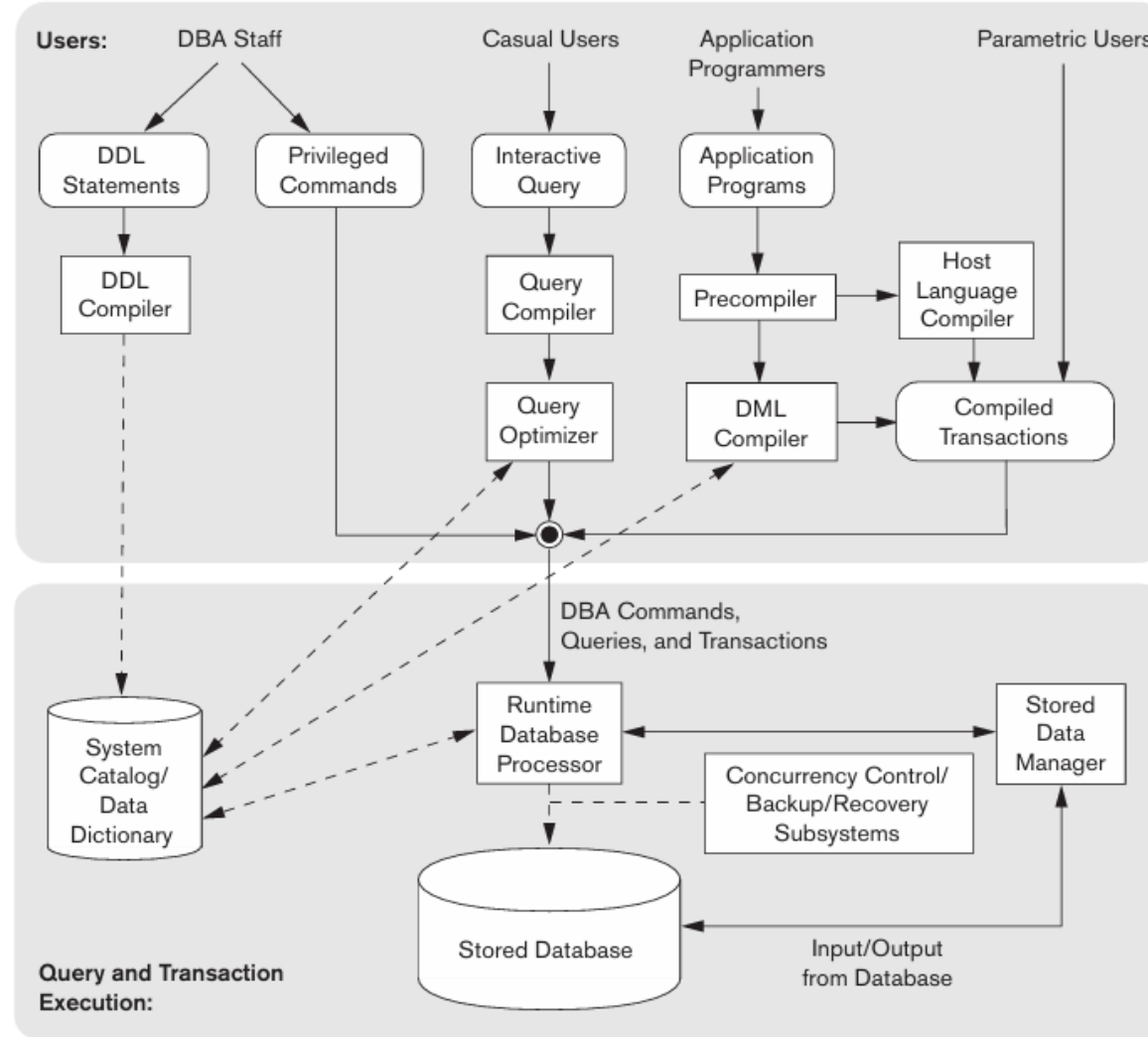


Figure 2.3
Component modules of a DBMS and their interactions.

The figure is divided into two parts.

- The top part of the figure refers to the various users of the database environment and their interfaces.
- The lower part shows the internal modules of the DBMS responsible for storage of data and processing of transactions.
- The **database and DBMS catalog** are stored on disk; disk access is mainly controlled by the **Operating System (OS)**.
- Many DBMSs use their own **buffer manager** to reduce disk read/write and improve performance.
- The **Stored Data Manager** controls access to data and catalog stored on disk.

Users and Interfaces:

- **DBA staff:** Define and tune the database using DDL and privileged commands.
- **Casual users:** Use interactive query interfaces to retrieve data.
- **Application programmers:** Write programs in languages like Java, C, C++, Python, PHP.
- **Parametric users:** Execute predefined transactions by supplying parameters (e.g., bank transactions).

DBMS Processing Modules:

- **DDL Compiler:** Processes schema definitions and stores metadata in the system catalog.
- **System Catalog:** Stores schema info, file details, data types, constraints, and statistics.
- **Query Compiler & Optimizer:** Parses, validates, and optimizes queries for efficient execution.
- **DML Precompiler:** Extracts database commands from programs and sends them for compilation.
- **Runtime Database Processor:** Executes queries, transactions, and privileged commands; manages buffers and interacts with stored data manager.
- **Concurrency Control & Recovery:** Ensure transaction consistency, backup, and recovery.

Client-Server Architecture:

- **Client:** Runs user applications.
- **Database Server:** Stores and manages the database.
- **Application Server** (optional): Middle layer between client and database server.

Interaction with OS:

- OS schedules disk access in shared systems.
- DBMS may control memory buffering in dedicated database servers.
- DBMS interfaces with compilers, application servers, and network systems.

2. Database System Utilities

In addition to possessing the software modules, most DBMSs have database utilities that help the DBA manage the database system. Common utilities have the following types of functions:

Loading

- Used to **load existing data files** (text files, sequential files, etc.) into the database.
- Converts data from **source format** to **target database format automatically**.
- Helps in **data migration** from one DBMS to another.
- Some tools automatically generate loading programs using source and target schemas.

Backup

- Creates a **backup copy of the database** on tape or other storage devices.
- Used to **restore the database after disk failures or crashes**.
- **Incremental backup** stores only changes since the last backup, saving storage space but more complex to manage.

Database Storage Reorganization

- Reorganizes database files into **new storage structures**.
- Creates **new indexes or access paths** to improve performance.
- Helps reduce fragmentation and improve retrieval speed.

Performance Monitoring

- Monitors **database usage and performance statistics**.
- Helps DBA decide:
 - ✓ Whether to reorganize files
 - ✓ Whether to add or remove indexes
 - ✓ How to tune the database for better performance

Other utilities may be available for sorting files, handling data compression, monitoring access by users, interfacing with the network and performing other functions.

Tools, Application Environments and Communications Facilities

CASE Tools (Computer-Aided Software Engineering Tools)

- Used during the **database design phase**.
- Help designers in:
 - Database modeling
 - Schema design
 - Documentation and system planning

Data Dictionary / Information Repository

- An **expanded data dictionary** stores more information than the DBMS catalog.
- It includes:
 - Schema and constraints
 - Design decisions
 - Usage standards
 - Application program descriptions
 - User information

Also called an **information repository**.

Accessed mainly by **users and DBA**, not just by DBMS software

Application Development Environments

- Tools like **PowerBuilder, JBuilder**, etc.
- Provide an environment for:
 - Database design
 - GUI development
 - Querying and updating data
 - Application program development

Communication Software and DB/DC Systems

- DBMS must interface with **communication software** to allow **remote users** to access databases.
- Remote access uses:
 - Internet routers
 - Phone lines
 - LAN/WAN networks
 - Satellite communication
- The integrated **Database + Data Communication system** is called a **DB/DC system**.

Centralized and Client/Server Architectures for DBMSs

Centralized DBMSs Architecture

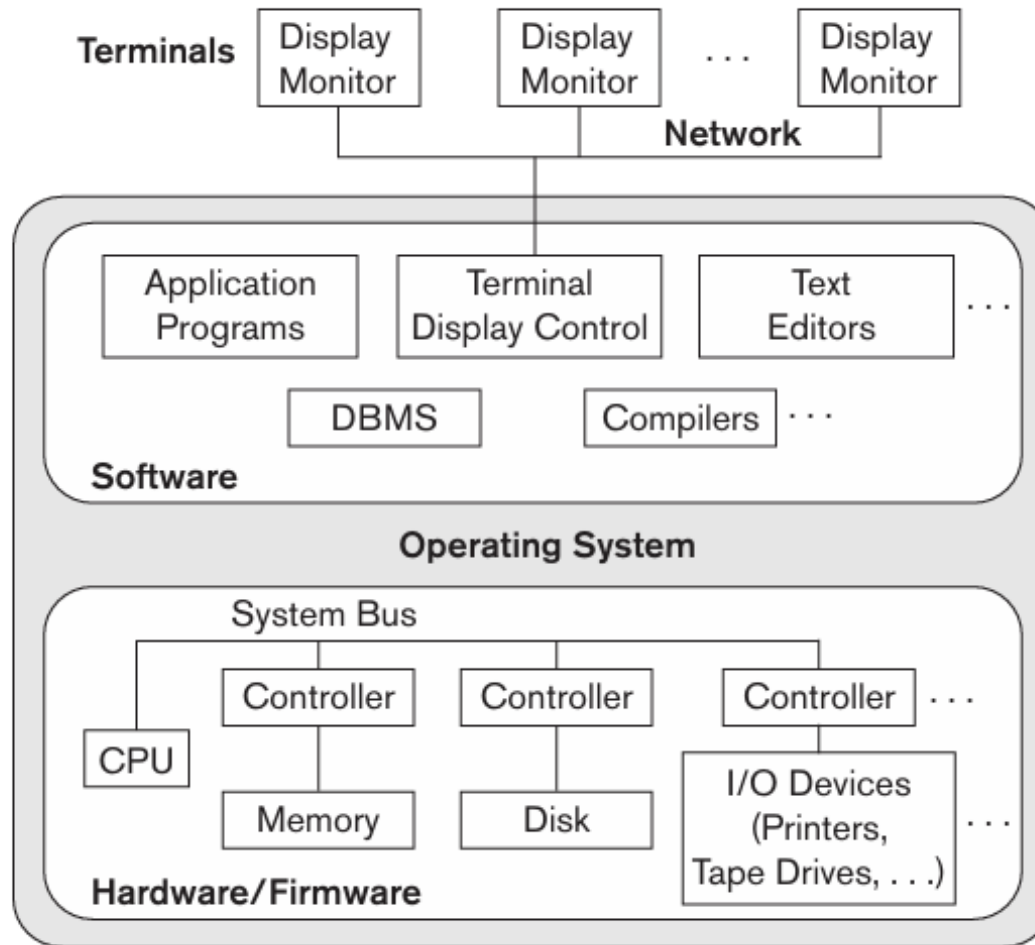


Figure 2.4
A physical centralized architecture.

- Early DBMS architectures were **centralized** and ran on mainframe computers.
- The mainframe handled everything:
 - ✓ DBMS functionality
 - ✓ Application program execution
 - ✓ User interface processing
- Users accessed the DBMS using dumb terminals (no processing power).
- Terminals only provided:
 - ✓ Display of results
 - ✓ Input of commands
- All processing was done remotely on the central machine.
- Communication occurred through networks connecting terminals to the mainframe.

- As hardware prices decreased, terminals were replaced by:
 - ✓ PCs
 - ✓ Workstations
 - ✓ Mobile devices
- Initially, these devices were still used like terminals, so the DBMS remained centralized.
- Over time, DBMSs began to use the processing power at the user side.
- This shift led to the development of client/server DBMS architectures, where:
 - ✓ Some processing is done on the client
 - ✓ DBMS services are shared between client and server

Basic Client/Server Architectures

- Client/server architecture was developed for environments with:
 - ✓ Many PCs, workstations, and mobile devices
 - ✓ Fewer specialized server machines
 - ✓ All connected through LANs, wireless networks, or other networks
- The main idea is to use specialized servers, each with a specific function, such as:
 - ✓ File servers (store and manage files for clients)
 - ✓ Printer servers (handle print requests)
 - ✓ Database servers
 - ✓ Web servers
 - ✓ E-mail servers

Clients:

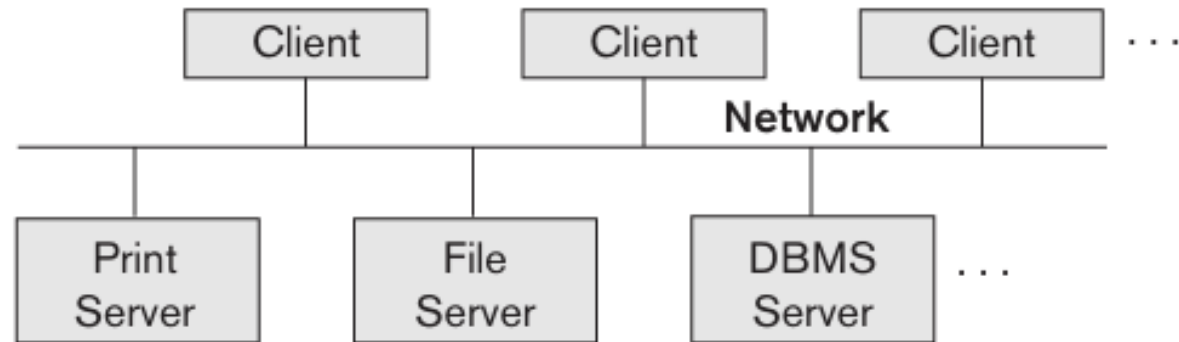
- Are typically PCs, workstations, or mobile devices
- Provide the user interface
- Perform local processing
- Send requests to servers when they need services not available locally

Servers:

- Contain hardware and software to provide services to clients
- Offer shared resources like:
 - ✓ File access
 - ✓ Printing
 - ✓ Archiving
 - ✓ Database access
- Can serve many clients simultaneously

Two-Tier Client/Server Architectures for DBMSs

Figure 2.5
Logical two-tier
client/server
architecture.



- Many RDBMSs initially started as centralized systems.
- The first components moved to the client side were:
 - ✓ User interface programs
 - ✓ Application programs
- SQL provided a clear logical boundary between client and server.
- As a result:
 - ✓ Query processing remained on the server
 - ✓ Transaction management remained on the server
- The server in this architecture is called:
 - ✓ Query server
 - ✓ Transaction server
 - ✓ Commonly an SQL server

- On the client side:
 - ✓ User interface and application programs run locally
 - ✓ Programs connect to the DBMS when database access is required
- After connection:
 - ✓ Clients send SQL queries and transaction requests to the server
 - ✓ Server processes them and sends results back to the client
- ODBC (Open Database Connectivity):
 - ✓ Standard API for database access
 - ✓ Allows client programs to communicate with DBMSs
- JDBC:
 - ✓ Java-based standard for DBMS access
 - ✓ Allows Java client programs to access one or more DBMSs

- This setup is called a two-tier architecture because:
Software is split between client and server only
- Advantages of two-tier architecture:
 - ✓ Simple design
 - ✓ Easy integration with existing systems
- Growth of the Web changed client–server roles and led to: Three-tier architecture

Three-Tier Architectures for Web Applications

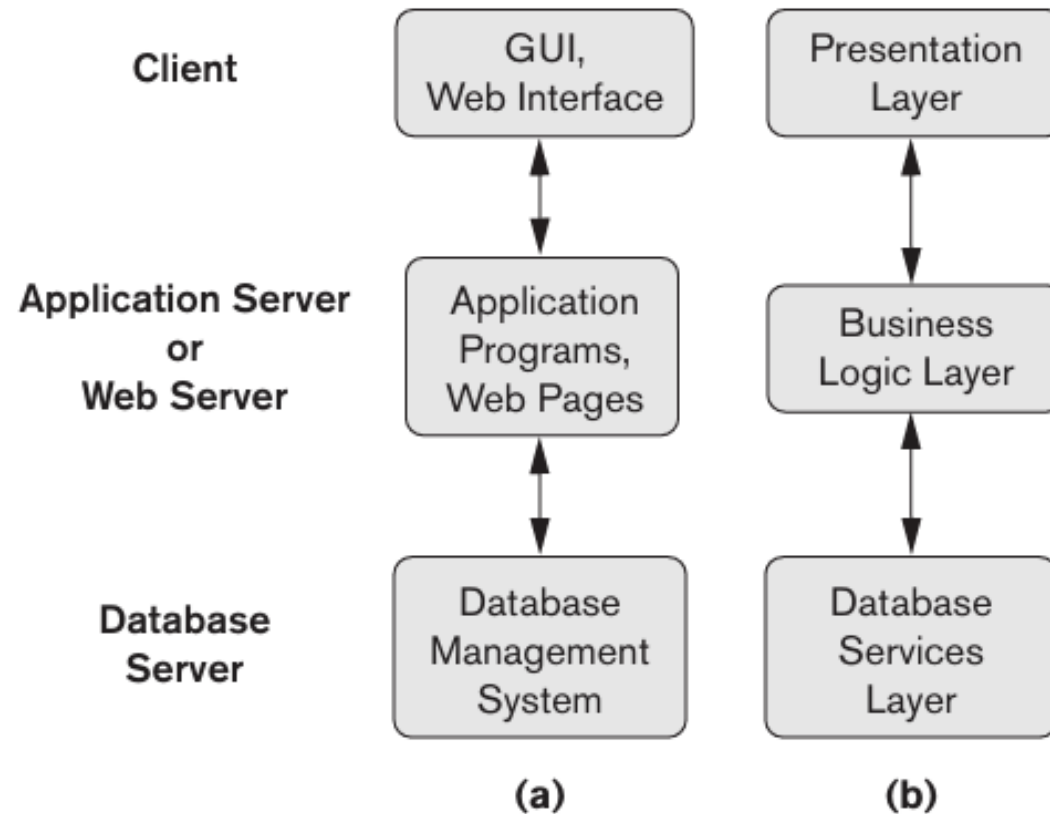


Figure 2.7
 Logical three-tier
 client/server
 architecture, with a
 couple of commonly
 used nomenclatures.

- Three-tier architecture is widely used in Web applications.
- It introduces an intermediate layer between: Client and Database server
- The middle tier is called: Application server or Web server (depending on the application)
- Roles of the middle tier:
 - ✓ Runs application programs
 - ✓ Stores and enforces business rules (procedures and constraints)
 - ✓ Acts as an intermediary between client and database server
 - ✓ Improves database security by verifying client credentials
 - ✓ Sends queries/commands to the database server
 - ✓ Returns processed results to the client

Three tiers in terms of functionality:

- ✓ Presentation tier – user interface and data entry
- ✓ Business logic tier – application rules and constraints
- ✓ Data tier – database and data management services

The middle tier may also:

- ✓ Act as a Web server
- ✓ Format query results into dynamic Web pages
- ✓ Deliver them to the client's Web browser

Three-tier architecture provides:

- ✓ Better scalability
- ✓ Improved security
- ✓ Clear separation of responsibilities

Data Modeling Using the Entity Relationship (ER) Model

Using High-Level Conceptual Data Models for Database Design

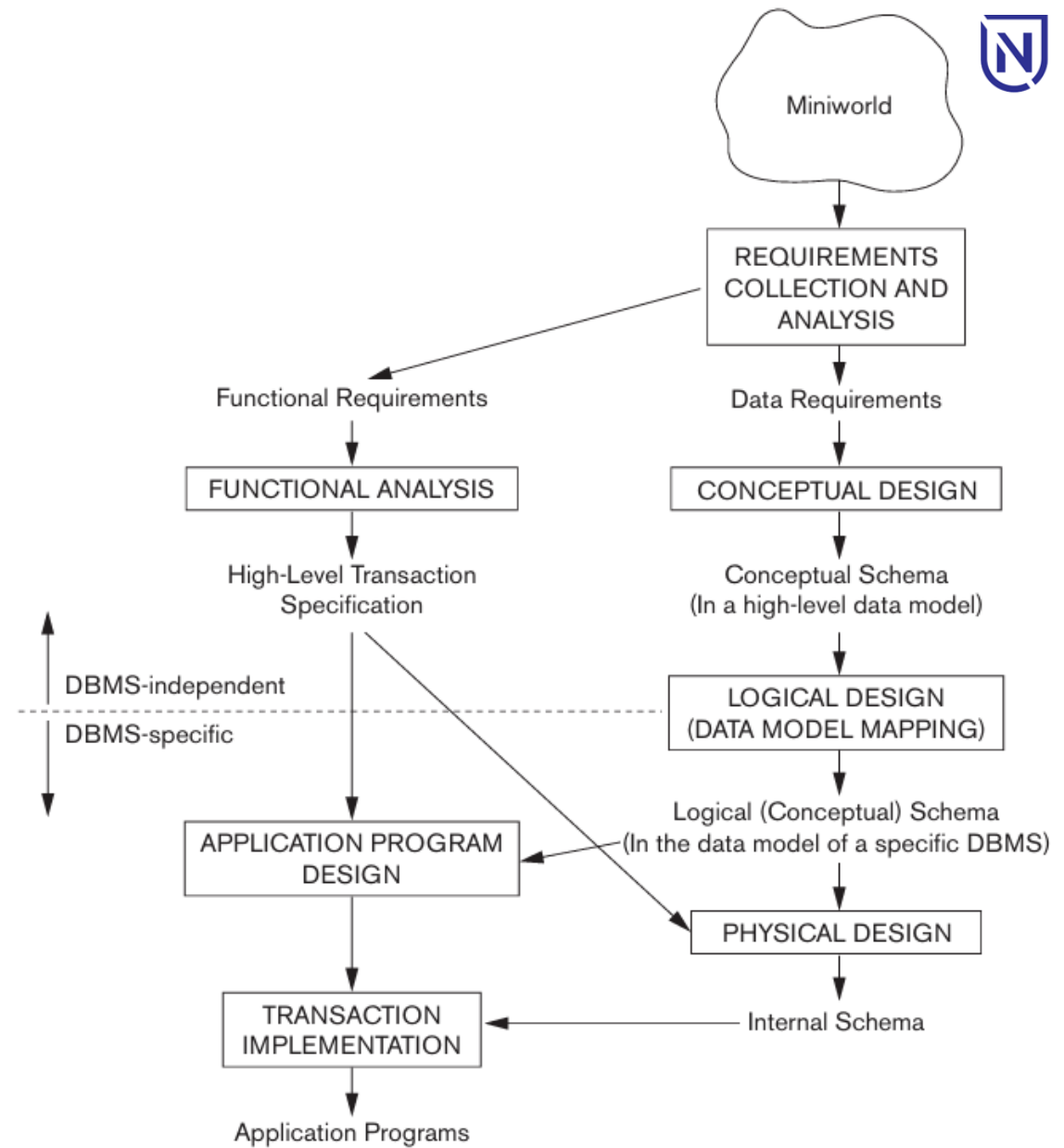


Figure 3.1

A simplified diagram to illustrate the main phases of database design.

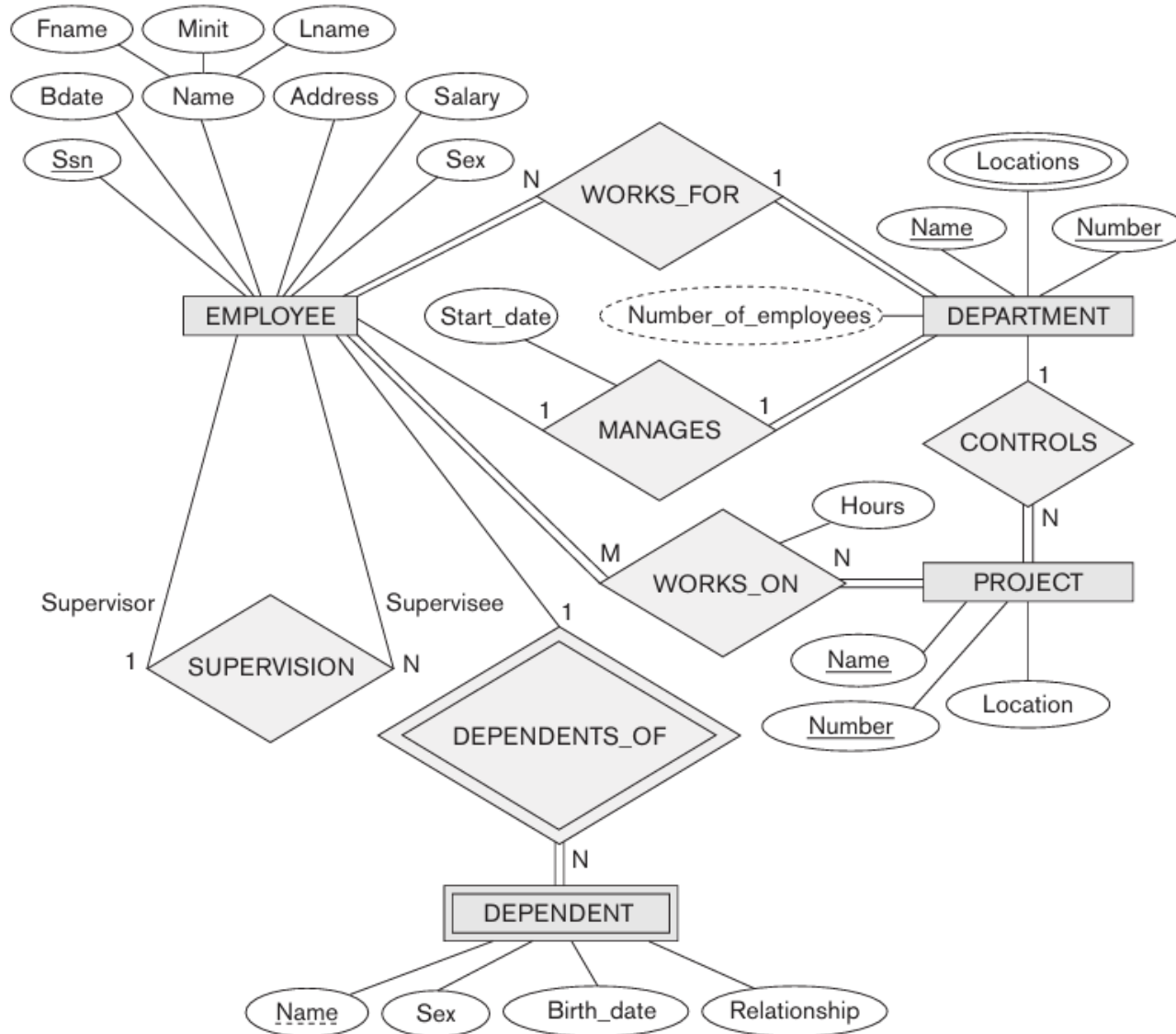
Steps in database design process

- The first step is **requirements collection and analysis**. During this step, the database designers interview database users to understand and document their data requirements. The result of this step is a concisely written set of users' requirements. These requirements should be specified in as detailed and complete a form as possible.
- In parallel with specifying the **data requirements**, it is useful to specify the known **functional requirements** of the application. These consist of the user defined operations (or transactions) that will be applied to the database, including both retrievals and updates

- After gathering and analyzing requirements, the **conceptual design** phase creates a **conceptual schema** using a **high-level data model**. This schema describes the database in terms of **entities, relationships, and constraints**, without including implementation or storage details. Because it is easy to understand, it helps in **communicating with nontechnical users** and ensures that **all user requirements are complete and consistent**. Conceptual design allows database designers to focus on the **structure and meaning of data**, making it easier to develop an effective database design.

- After conceptual design, the database is **implemented using a commercial DBMS**. During this stage, the **conceptual schema** is converted into an **implementation data model** (commonly the **relational/SQL model**). This process is called **logical design** or **data model mapping**, and it results in a **database schema** compatible with the chosen DBMS. The mapping process is often **automated or semi-automated** using database design tools.
- The last step is the **physical design phase**, during which the internal storage structures, file organizations, indexes, access paths, and physical design parameters for the database files are specified. In parallel with these activities, **application programs** are designed and implemented as database transactions corresponding to the high-level transaction specifications.

A Sample Database Application - COMPANY



Entity Types, Entity Sets, Attributes and Keys

Entities and Attributes

- The basic concept that the ER model represents is an entity, which is a thing or object in the real world with an independent existence.
- An entity may be an object with a physical existence (for example, a particular person, car, house, or employee) or it may be an object with a conceptual existence (for instance, a company, a job, or a university course).
- Each entity has attributes—the particular properties that describe it. For example, an EMPLOYEE entity may be described by the employee's name, age, address, salary, and job.
- A particular entity will have a value for each of its attributes.
- The attribute values that describe each entity become a major part of the data stored in the database.

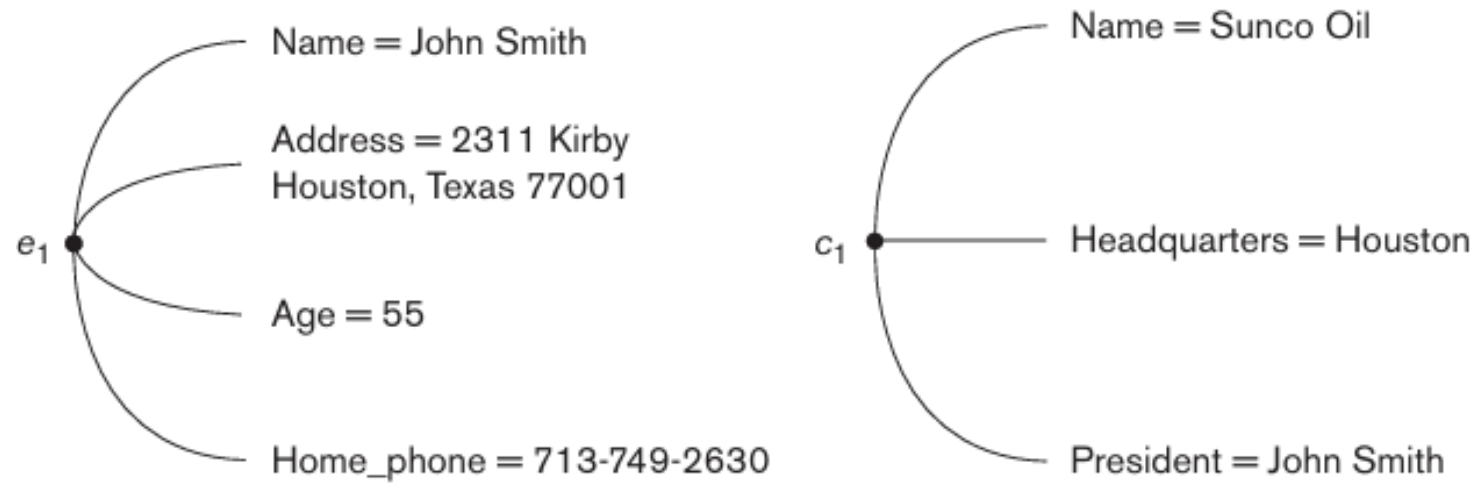


Figure 3.3

Two entities, EMPLOYEE e_1 , and COMPANY c_1 , and their attributes.

Attribute Types

Composite versus Simple (Atomic) Attributes

- **Composite attributes** can be divided into smaller subparts, which represent more basic attributes with independent meanings.
- For example, the Address attribute of the EMPLOYEE entity can be subdivided into Street_address, City, State and Zip with the values '2311 Kirby', 'Houston', 'Texas' and '77001'.
- Attributes that are not divisible are called **simple or atomic attributes**.
- Composite attributes can form a hierarchy; for example, Street_address can be further subdivided into three simple component attributes: Number, Street, and Apartment_number, as shown in Fig.

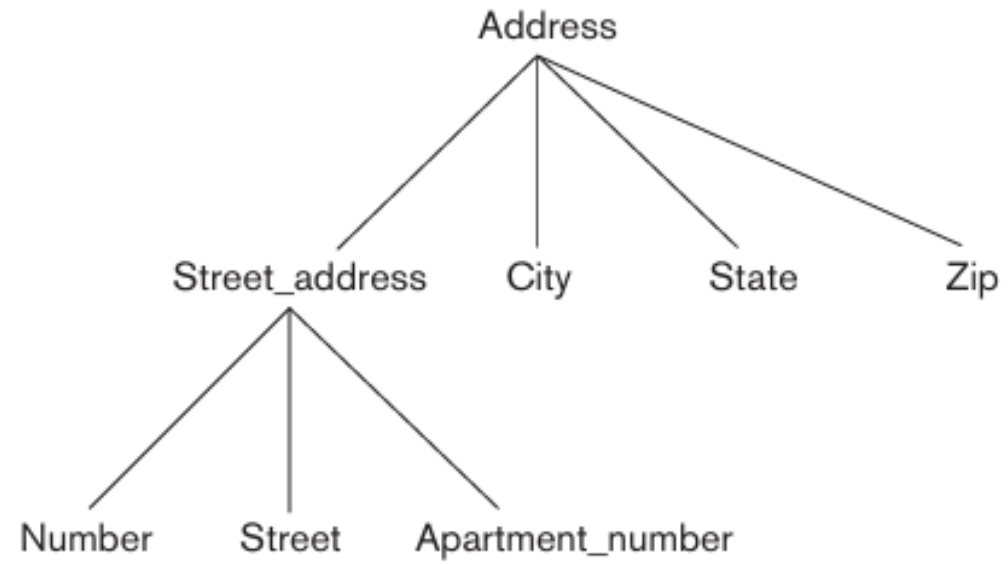


Figure 3.4
A hierarchy of
composite attributes.

Single-Valued versus Multivalued Attributes

- Most attributes have a single value for a particular entity; such attributes are called **single-valued**. For example, Age is a single-valued attribute of a person.
- In some cases, an attribute can have a set of values for the same entity—for instance, a Colors attribute for a car, or a College_degrees attribute for a person. Cars with one color have a single value, whereas two-tone cars have two color values. Similarly, one person may not have any college degrees, another person may have one, and a third person may have two or more degrees; therefore, different people can have different numbers of values for the College_degrees attribute. Such attributes are called **multivalued**.

Stored versus Derived Attributes

- In some cases, two (or more) attribute values are related—for example, the Age and Birth_date attributes of a person.
- For a particular person entity, the value of Age can be determined from the current (today's) date and the value of that person's Birth_date.
- The Age attribute is hence called a **derived attribute** and is said to be derivable from the Birth_date attribute, which is called a **stored attribute**.
- Some attribute values can be derived from related entities; for example, an attribute Number_of_employees of a DEPARTMENT entity can be derived by counting the number of employees working for that department

NULL Values

- In some cases, a particular entity may not have an applicable value for an attribute.
- For example, the Apartment_number attribute of an address applies only to addresses that are in apartment buildings and not to other types of residences, such as single-family homes.
- Similarly, a College_degrees attribute applies only to people with college degrees.
- For such situations, a special value called **NULL** is created.
- An address of a single-family home would have NULL for its Apartment_number attribute, and a person with no college degree would have NULL for College_degrees.

- NULL can also be used if we do not know the value of an attribute for a particular entity—for example, if we do not know the home phone number of ‘John Smith’.
- The meaning of the former type of NULL is **not applicable**, whereas the meaning of the latter is **unknown**.
- The **unknown category of NULL** can be further classified into two cases.
 - ✓ The first case arises when it is known that the attribute value exists but is **missing**—for instance, if the Height attribute of a person is listed as NULL.
 - ✓ The second case arises when it is **not known whether the attribute value exists**—for example, if the Home_phone attribute of a person is NULL.

Complex Attributes

- Composite and multivalued attributes can be nested arbitrarily.
- We can represent arbitrary nesting by grouping components of a composite attribute between parentheses () and separating the components with commas, and by displaying multivalued attributes between braces { }. Such attributes are called **complex attributes**.
- For example, if a person can have more than one residence and each residence can have a single address and multiple phones, an attribute Address_phone for a person can be specified as

```
{Address_phone( {Phone(Area_code,Phone_number)},Address(Street_address  
(Number,Street,Apartment_number),City,State,Zip) )}
```

Figure 3.5

A complex attribute:
Address_phone.

Entity Types, Entity Sets, Attributes and Keys

- An **entity type** defines a collection of entities that have the same attributes.
- Each entity type in the database is described by its name and attributes.
- Fig shows two entity types: EMPLOYEE and COMPANY, and a list of some of the attributes for each.

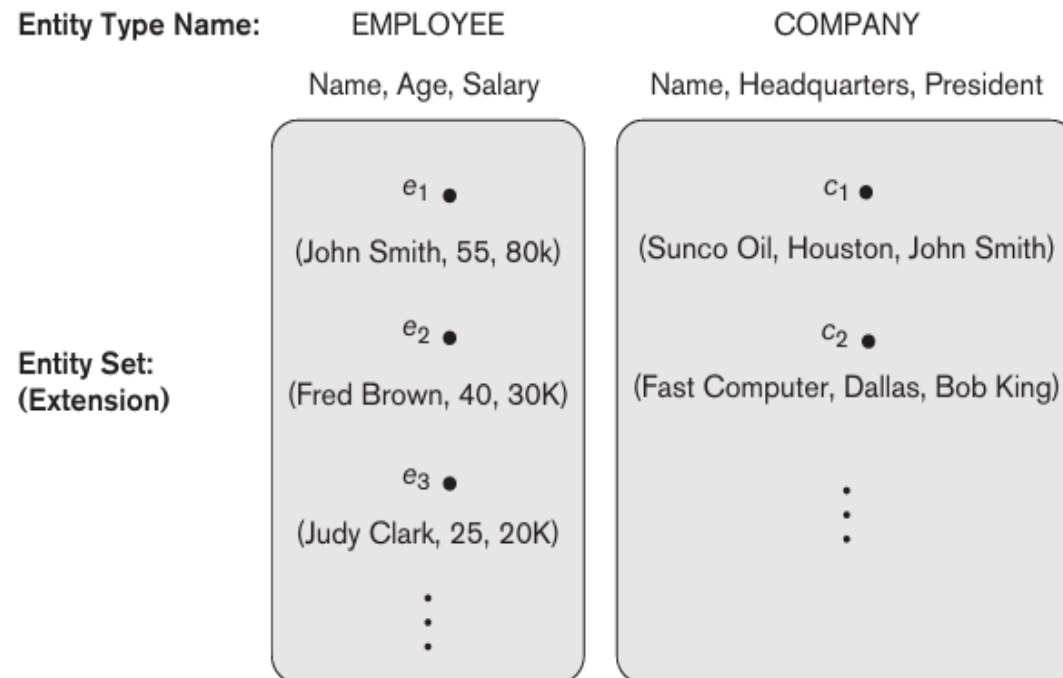


Figure 3.6
Two entity types, EMPLOYEE and COMPANY, and some member entities of each.

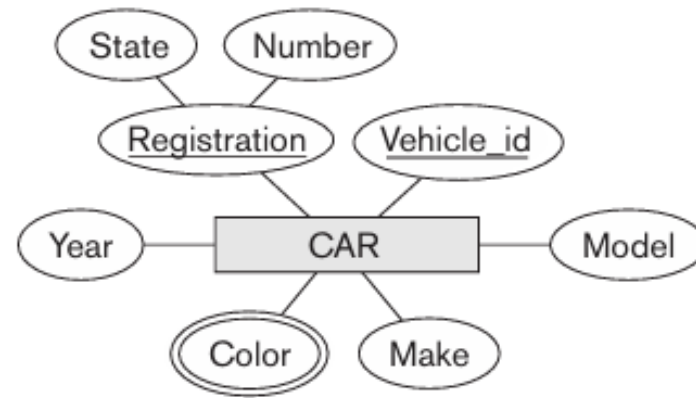
- The collection of all entities of a particular entity type in the database at any point in time is called an **entity set or entity collection**.
- The entity set is usually referred to using the same name as the entity type, even though they are two separate concepts.
- For example, EMPLOYEE refers to both a type of entity as well as the current collection of all employee entities in the database.

Key Attributes of an Entity Type

- An attribute of an entity type for which each entity must have a unique value is called a **key attribute** of the entity type.
- For example, for the PERSON entity type, a typical key attribute is SSN (Social Security Number).
- In ER diagrammatic notation, each key attribute has its name underlined inside the oval.
- Some entity types have more than one key attribute. For example, each of the Vehicle_id and Registration attributes of the entity type CAR is a key in its own right.

- The Registration attribute is an example of a composite key formed from two simple component attributes, State and Number, neither of which is a key on its own.
- An entity type may also have no key, in which case it is called a **weak entity** type.
- If two attributes are underlined separately, then each is a key on its own.

(a)



(b)

CAR
 Registration (Number, State), Vehicle_id, Make, Model, Year, {Color}

CAR₁
 ((ABC 123, TEXAS), TK629, Ford Mustang, convertible, 2004 {red, black})

CAR₂
 ((ABC 123, NEW YORK), WP9872, Nissan Maxima, 4-door, 2005, {blue})

CAR₃
 ((VSY 720, TEXAS), TD729, Chrysler LeBaron, 4-door, 2002, {white, blue})

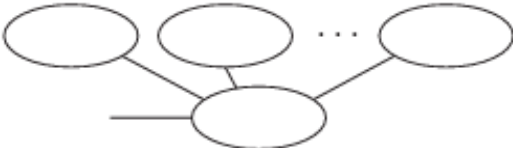
⋮

Figure 3.7

The CAR entity type with two key attributes, Registration and Vehicle_id. (a) ER diagram notation. (b) Entity set with three entities.

- An entity type is represented in ER diagrams as a **rectangular box** enclosing the entity type name.
- Attribute names are enclosed in **ovals** and are attached to their entity type by straight lines.
- Composite attributes are attached to their component attributes by straight lines.
- Multivalued attributes are displayed in **double ovals**.
- An entity type describes the **schema or intension** for a set of entities that share the same structure.
- The collection of entities of a particular entity type is grouped into an entity set, which is also called the **extension** of the entity type.

Notations for ER Diagram

Symbol	Meaning
	Entity
	Relationship
	Attribute
	Key Attribute
	Multivalued Attribute
	Composite Attribute

Initial Conceptual Design of the COMPANY Database

- Based on the requirements, we can identify four initial entity types in the COMPANY database:
 - ✓ DEPARTMENT
 - ✓ PROJECT
 - ✓ EMPLOYEE
 - ✓ DEPENDENT
- Their initial design is shown on the following slide
- The initial attributes shown are derived from the requirements description

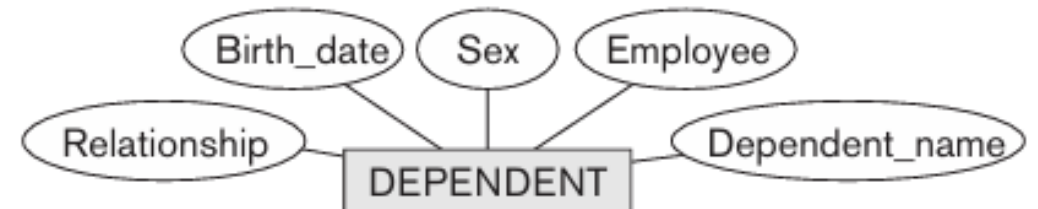
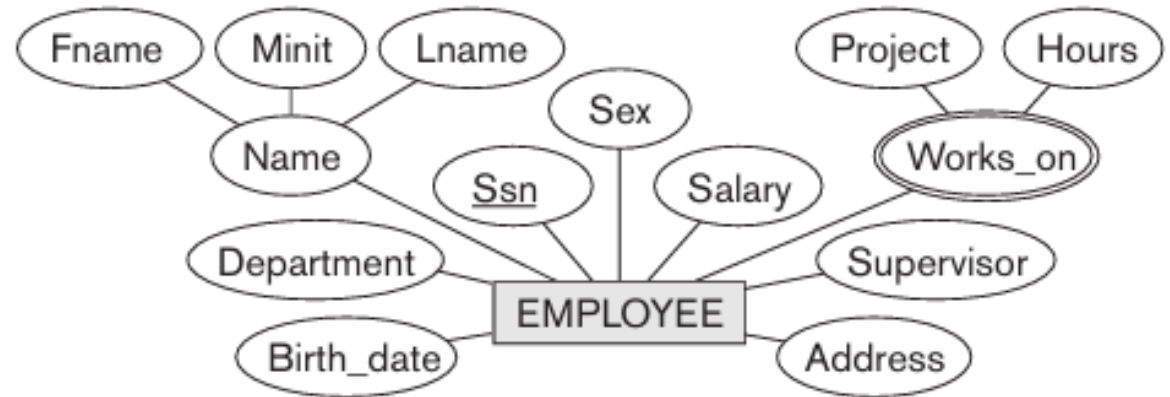
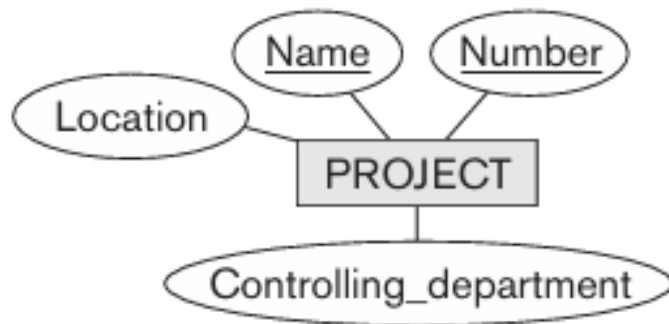
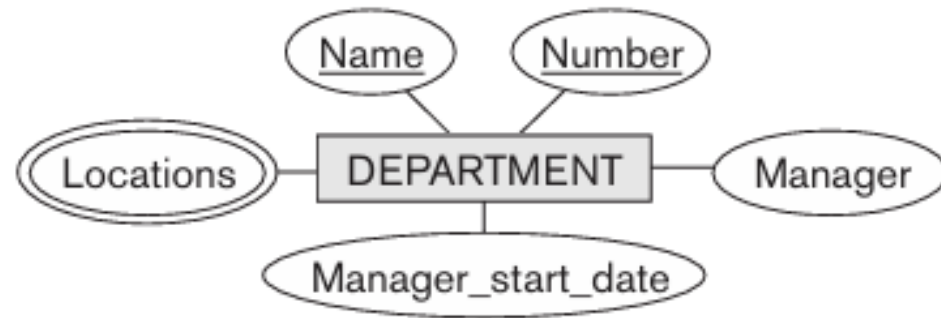


Figure 3.8

Preliminary design of entity types for the COMPANY database. Some of the shown attributes will be refined into relationships.

Refining the initial design by introducing relationships

- The initial design is typically not complete
- Some aspects in the requirements will be represented as **relationships**

ER model has three main concepts:

- ✓ Entities (and their entity types and entity sets)
- ✓ Attributes (simple, composite, multivalued)
- ✓ Relationships (and their relationship types and relationship sets)

Relationships and Relationship Types

- A **relationship** relates two or more distinct entities with a specific meaning.
- For example, EMPLOYEE John ***WORKS_ON*** the ProductX of PROJECT, or EMPLOYEE Franklin ***MANAGES*** the Research DEPARTMENT.
- Relationships of the same type are grouped or typed into a **relationship type**.
- For example, the WORKS_ON relationship type in which EMPLOYEEs and PROJECTs participate, or the MANAGES relationship type in which EMPLOYEEs and DEPARTMENTs participate.
- The **degree of a relationship type** is the number of participating entity types.
- A relationship type of degree two is called **binary**, and one of degree three is called **ternary**
- Both MANAGES and WORKS_ON are *binary* relationships

Relationship instances of the WORKS_FOR N:1 relationship between EMPLOYEE and DEPARTMENT

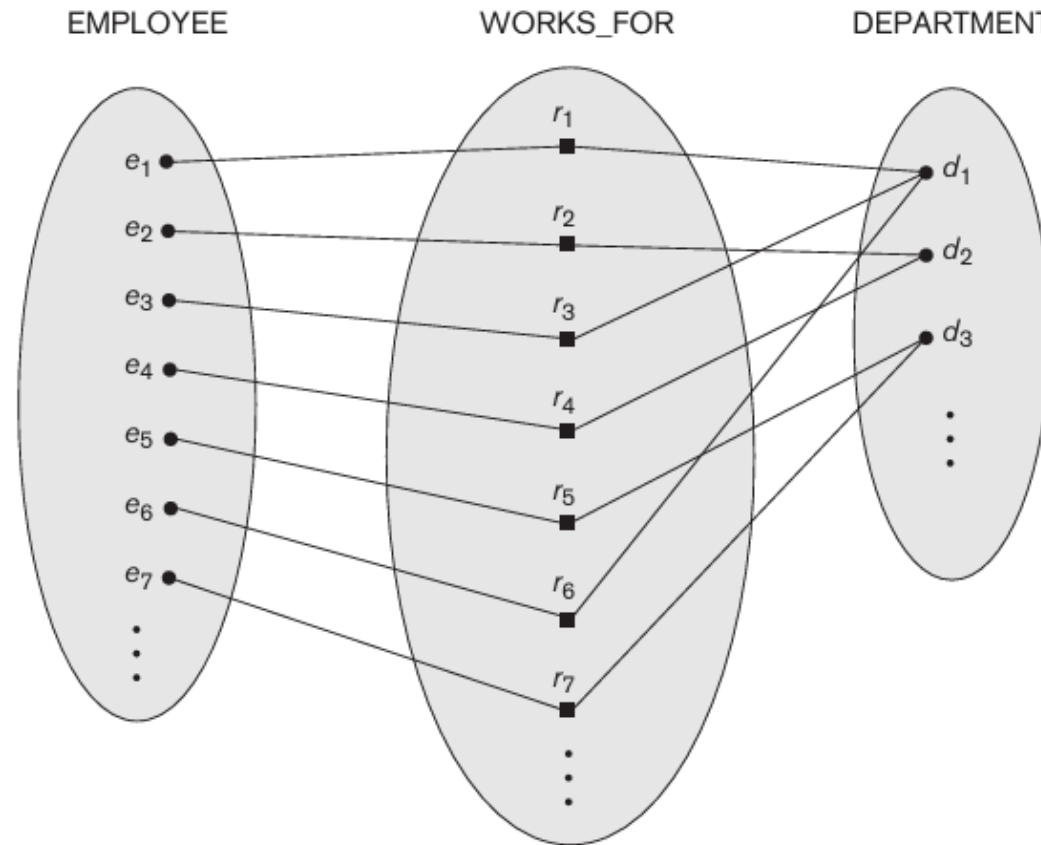


Figure 3.9

Some instances in the WORKS_FOR relationship set, which represents a relationship type WORKS_FOR between EMPLOYEE and DEPARTMENT.

Relationship instances of the M:N WORKS_ON relationship between EMPLOYEE and PROJECT

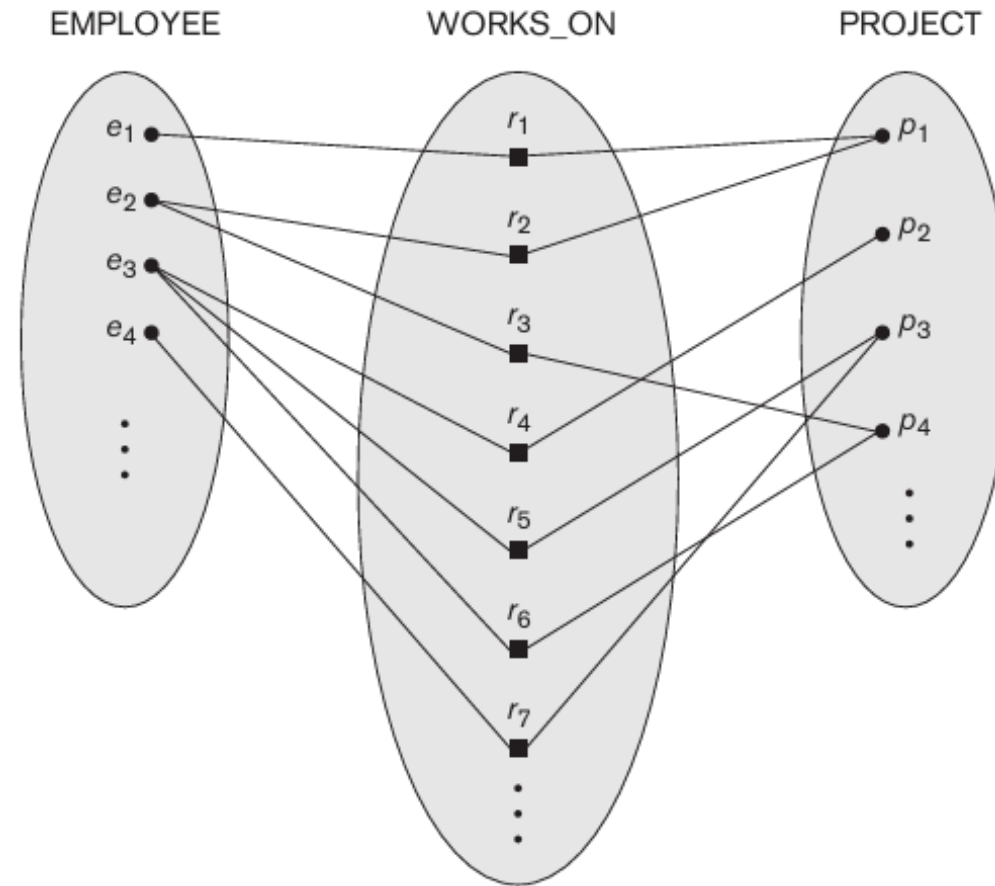


Figure 3.13
 An M:N relationship,
 WORKS_ON.

Refining the COMPANY database schema by introducing relationships

- By examining the requirements, six relationship types are identified
- All are *binary* relationships(degree 2)
- Listed below with their participating entity types:
 - ✓ WORKS_FOR (between EMPLOYEE, DEPARTMENT)
 - ✓ MANAGES (between EMPLOYEE, DEPARTMENT)
 - ✓ CONTROLS (between DEPARTMENT, PROJECT)
 - ✓ WORKS_ON (between EMPLOYEE, PROJECT)
 - ✓ SUPERVISION (between EMPLOYEE (as subordinate), EMPLOYEE (as supervisor))
 - ✓ DEPENDENTS_OF (between EMPLOYEE, DEPENDENT)

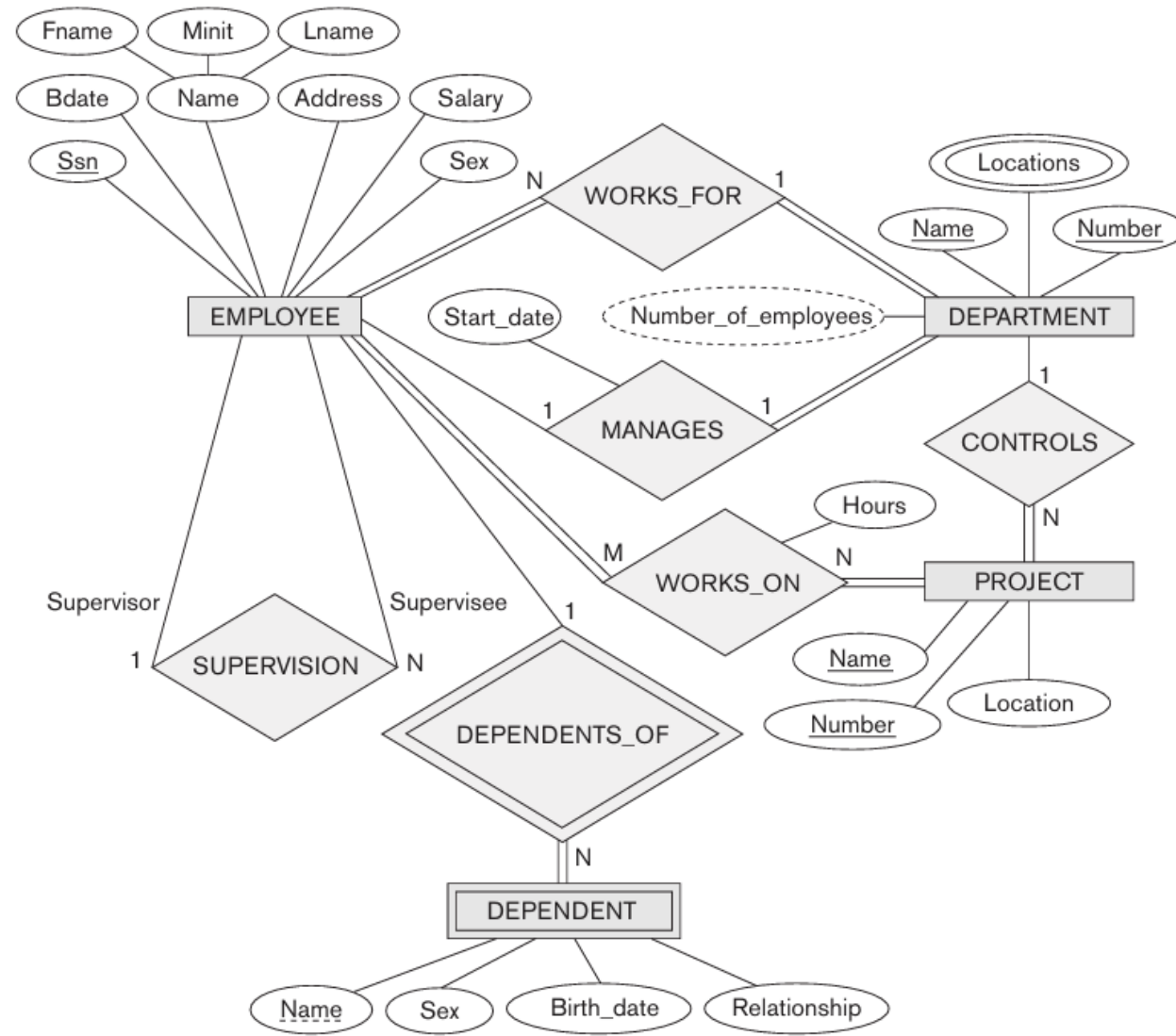


Figure 3.2

An ER schema diagram for the COMPANY database. The diagrammatic notation is introduced gradually throughout this chapter and is summarized in Figure 3.14.

In general, more than one relationship type can exist between the same participating entity types

MANAGES and WORKS_FOR are distinct relationship types between EMPLOYEE and DEPARTMENT

Different meanings and different relationship instances.

Relationship Type vs Relationship Set

Relationship Type:

- Is the schema description of a relationship
- Identifies the relationship name and the participating entity types
- Also identifies certain relationship constraints

Relationship Set:

- The current set of relationship instances represented in the database
- The current *state* of a relationship type

- Previous figures displayed the relationship sets
- Each instance in the set relates individual participating entities – one from each participating entity type
- In ER diagrams, we represent the *relationship type* as follows:
 - ✓ Diamond-shaped box is used to display a relationship type
 - ✓ Connected to the participating entity types via straight lines

Recursive Relationship Type

In some cases, the same entity type participates more than once in a relationship type in different roles. Such relationship types are called **recursive relationships or self-referencing relationships**.

Example: the SUPERVISION relationship - EMPLOYEE participates twice in two distinct roles

- ✓ supervisor (or boss) role
- ✓ supervisee (or subordinate) role

In following figure, first role participation labeled with 1 and second role participation labeled with 2.

In ER diagram, need to display role names to distinguish participations.

A Recursive Relationship Supervision

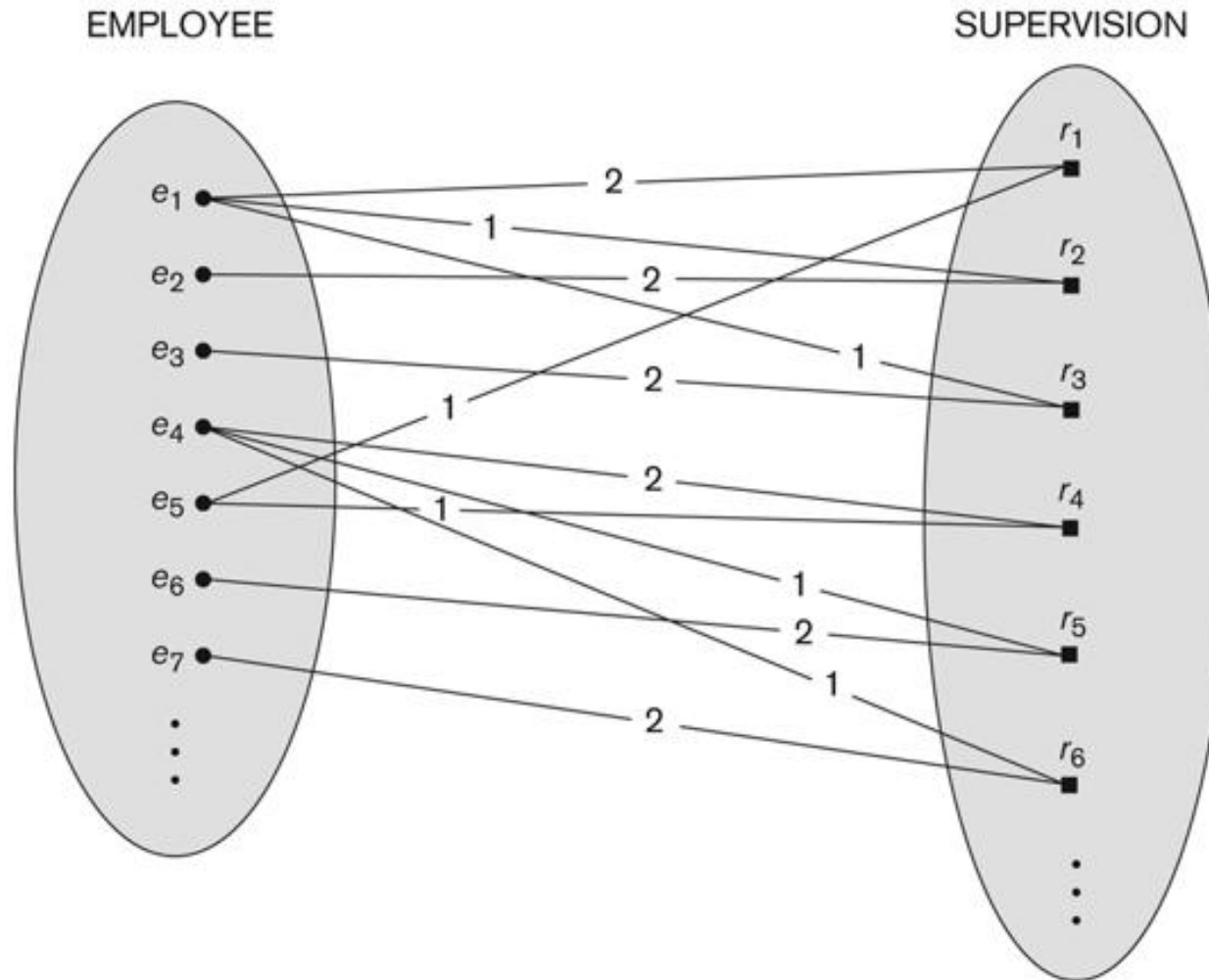


Figure 3.11

A recursive relationship SUPERVISION between EMPLOYEE in the *supervisor* role (1) and EMPLOYEE in the *subordinate* role (2).

Constraints on Relationships

Two main types of binary relationship constraints:

- **Cardinality Ratio** (specifies *maximum* participation)
- **Participation or Existence Dependency Constraint** (specifies *minimum* participation)

Cardinality Ratio

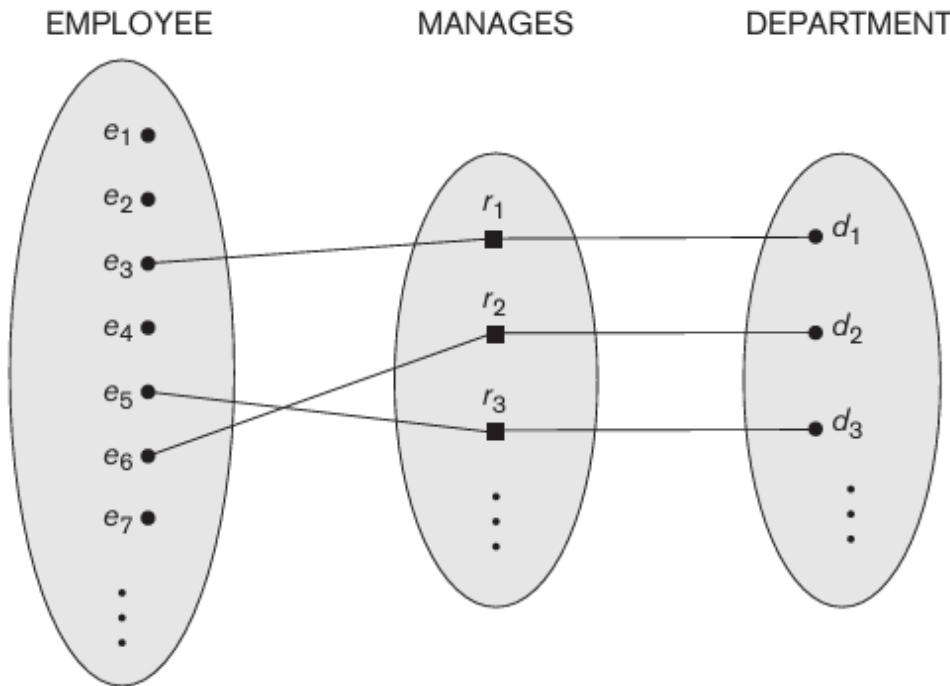
The cardinality ratio for a binary relationship specifies the maximum number of relationship instances that an entity can participate in.

- One-to-one (1:1)
- One-to-many (1:N) or Many-to-one (N:1)
- Many-to-many (M:N)

One-to-One (1:1) Relationship

Figure 3.12

A 1:1 relationship,
MANAGES.



An example of a 1:1 binary relationship is MANAGES, which relates a department entity to the employee who manages that department.

Many-to-one (N:1) Relationship

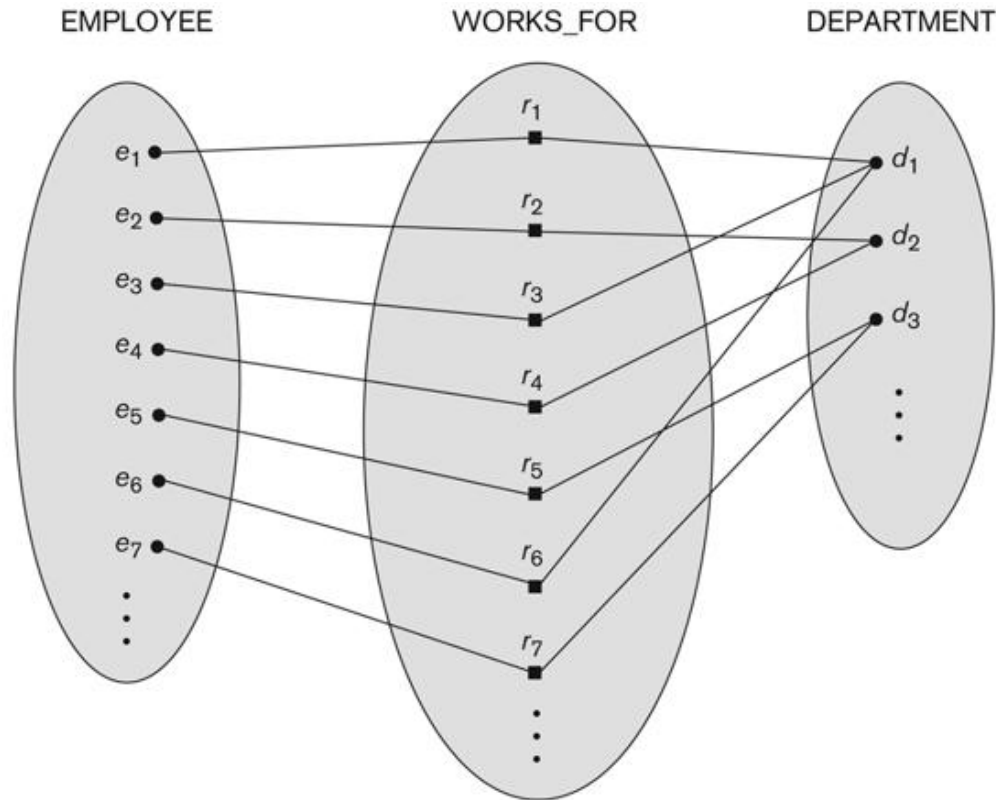


Figure 3.9

Some instances in the WORKS_FOR relationship set, which represents a relationship type WORKS_FOR between EMPLOYEE and DEPARTMENT.

An example of a N:1 binary relationship is WORKS_FOR, where many employees can work for one department

Many-to-Many (M:N) Relationship

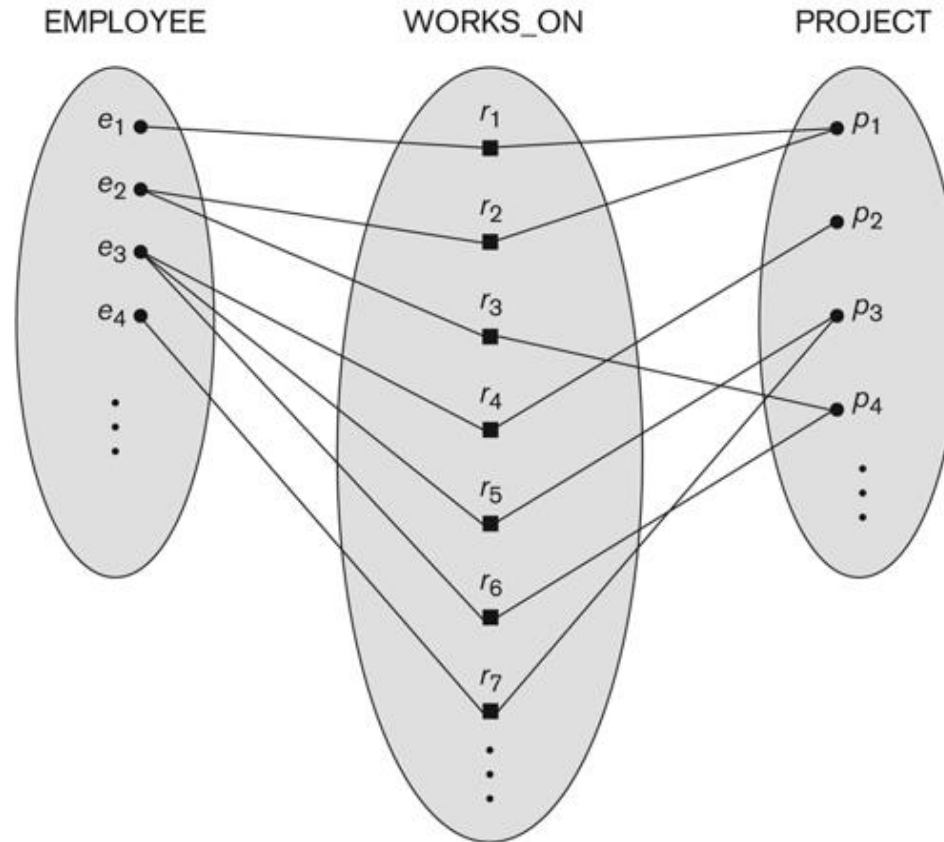


Figure 3.13
 An M:N relationship,
 WORKS_ON.

An example of a M:N binary relationship is WORKS_ON, where an employee can work on several projects and a project can have several employees.

Participation or Existence Dependency Constraint

- The participation constraint specifies whether the existence of an entity depends on its being related to another entity via the relationship type.
- This constraint specifies the minimum number of relationship instances that each entity can participate in and is sometimes called the **minimum cardinality constraint**.
- There are two types of participation constraints
 - ✓ Total
 - ✓ Partial

- If a company policy states that every employee must work for a department, then an employee entity can exist only if it participates in at least one WORKS_FOR relationship instance. Thus, the participation of EMPLOYEE in WORKS_FOR is called total participation, meaning that every entity in the total set of employee entities must be related to a department entity via WORKS_FOR. Total participation is also called **existence dependency**.
- We do not expect every employee to manage a department, so the participation of EMPLOYEE in the MANAGES relationship type is partial, meaning that some or part of the set of employee entities are related to some department entity via MANAGES, but not necessarily all.
- In ER diagrams, total participation is displayed as a **double line** connecting the participating entity type to the relationship, whereas partial participation is represented by a **single line**

Attributes of Relationship Types

- Relationship types can also have attributes, like those of entity types.
- For example, to record the number of hours per week that a particular employee works on a particular project, we can include an attribute **Hours** for the **WORKS_ON** relationship type.
- A value of **Hours** depends on a particular (employee, project) combination
- Another example is to include the date on which a manager started managing a department via an attribute **Start_date** for the **MANAGES** relationship type.

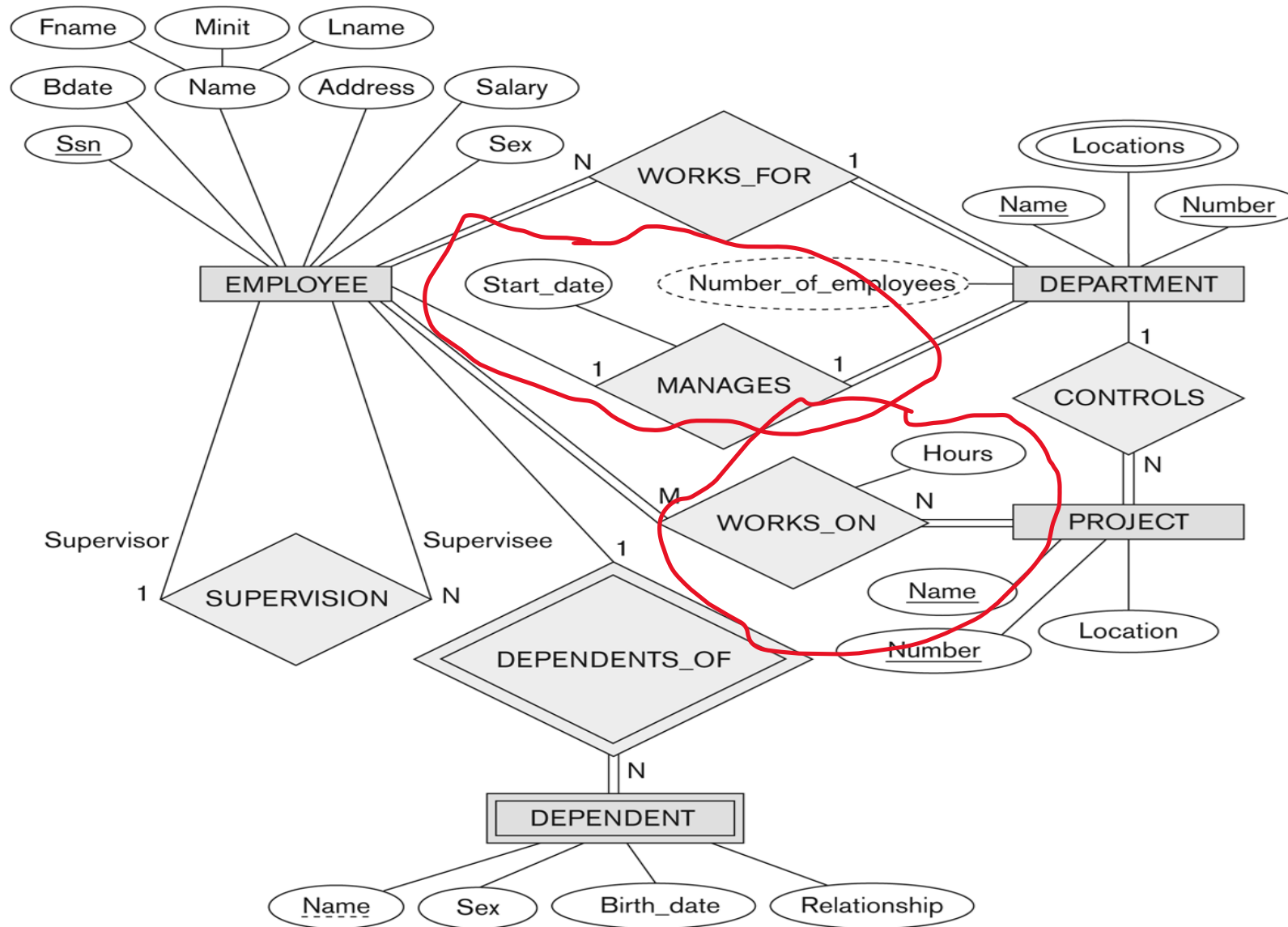


Figure 3.2

An ER schema diagram for the COMPANY database. The diagrammatic notation is introduced gradually throughout this chapter.

Weak Entity Types

- Entity types that do not have key attributes of their own are called weak entity types.
- In contrast, regular entity types that do have a key attribute are called strong entity types.
- A weak entity must participate in an **identifying relationship type** with an owner or **identifying entity type**
- A weak entity type normally has a **partial key**, which is the attribute that can uniquely identify weak entities that are related to the same owner entity.
- In ER diagrams, both a weak entity type and its identifying relationship are distinguished by surrounding their boxes and diamonds with double lines. The partial key attribute is underlined with a dashed or dotted line.

Weak Entities are identified by the combination of:

- A partial key of the weak entity type
- The particular entity they are related to in the identifying entity type

Example:

- A DEPENDENT entity is identified by the dependent's first name,
- and the specific EMPLOYEE with whom the dependent is related
- Name of DEPENDENT is the *partial key*
- DEPENDENT is a *weak entity type*
- EMPLOYEE is its identifying entity type via the identifying relationship type DEPENDENT_OF

Alternative (min, max) notation for relationship structural constraints

- Specified on each participation of an entity type E in a relationship type R
- Specifies that each entity e in E participates in at least *min* and at most *max* relationship instances in R
- Default(no constraint): $\text{min}=0$, $\text{max}=\infty$ (signifying no limit)
- Must have $\text{min} \leq \text{max}$, $\text{min} \geq 0$, $\text{max} \geq 1$

Examples:

A department has exactly one manager and an employee can manage at most one department.

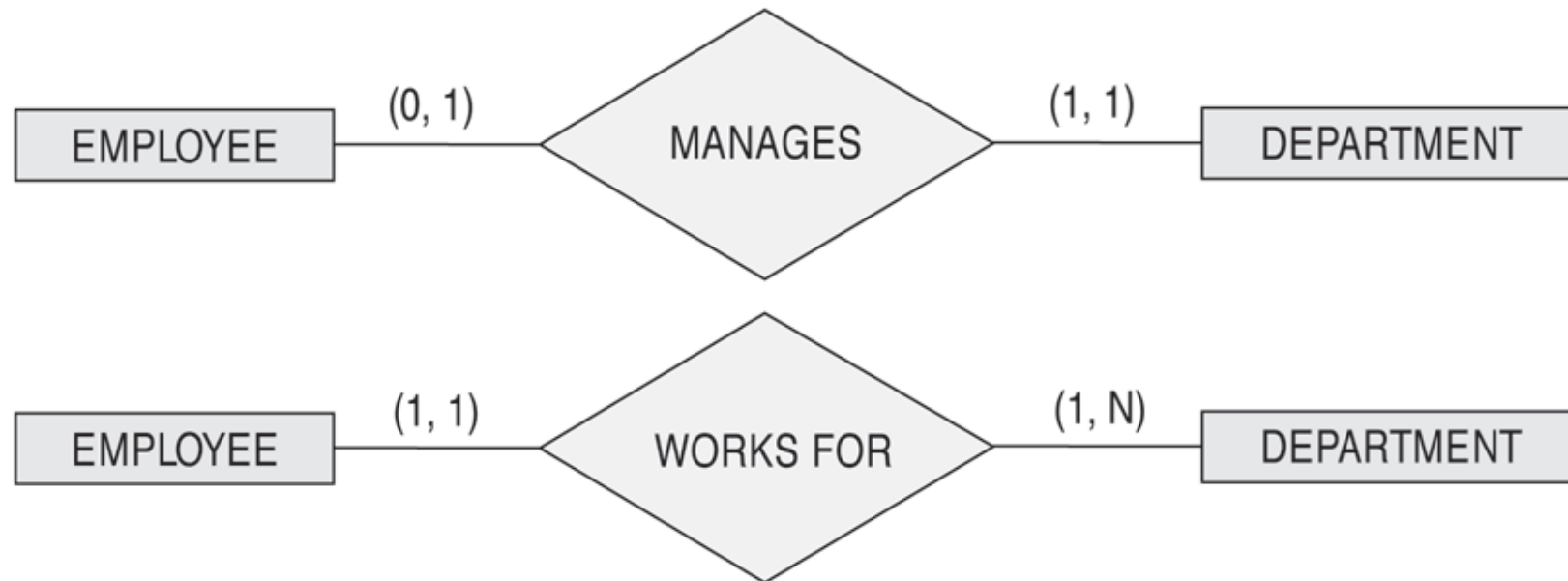
Specify (0,1) for participation of EMPLOYEE in MANAGES

Specify (1,1) for participation of DEPARTMENT in MANAGES

An employee can work for exactly one department, but a department can have any number of employees.

Specify (1,1) for participation of EMPLOYEE in WORKS_FOR
Specify (1,N) for participation of DEPARTMENT in WORKS_FOR

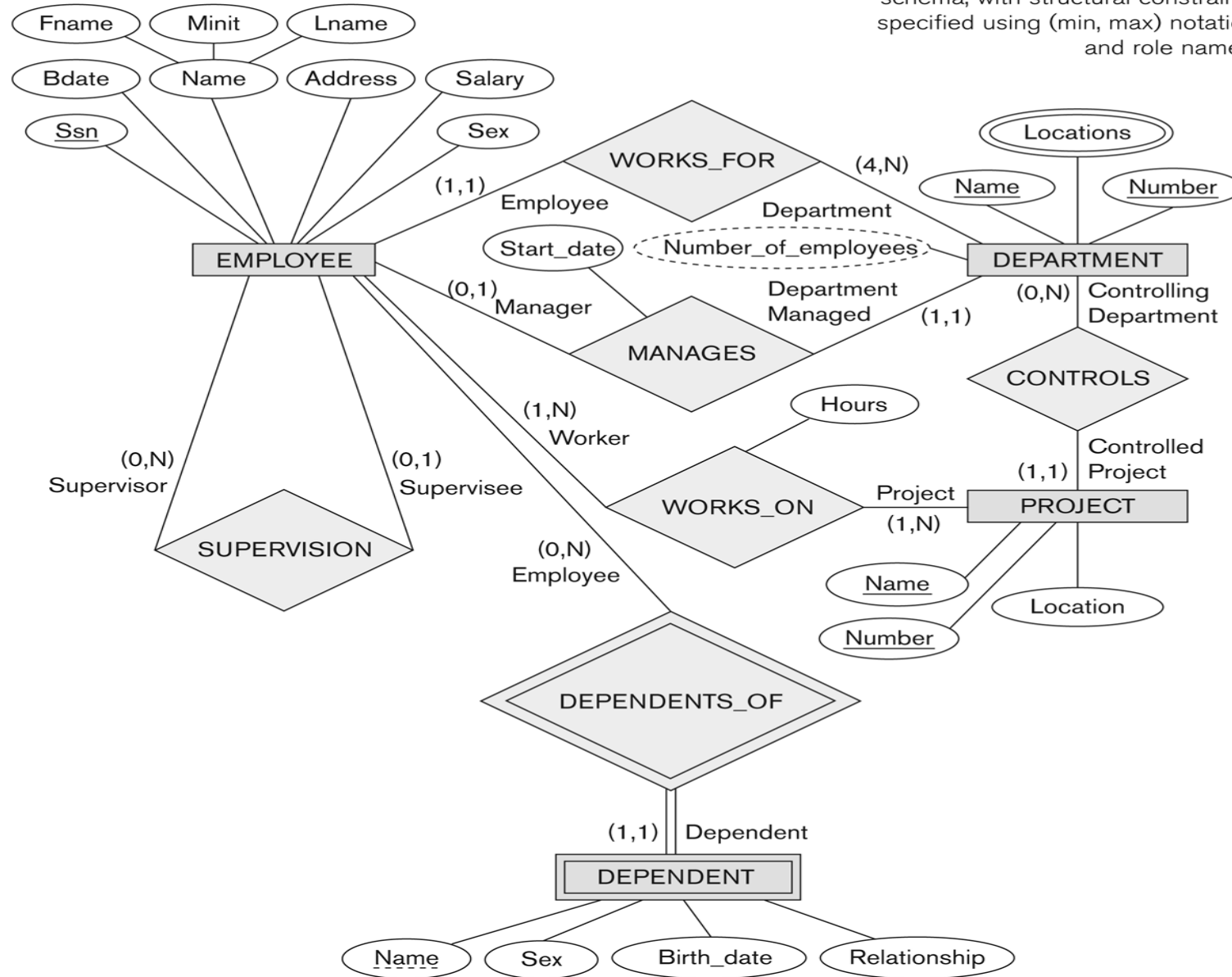
The (min,max) notation for relationship constraints



Read the min,max numbers next to the entity type and looking **away from** the entity type

Figure 3.15

ER diagrams for the company schema, with structural constraints specified using (min, max) notation and role names.



Practice Question 1

Draw an **Entity–Relationship (ER) diagram** for a **Student Course Enrollment System** with the following requirements:

- Each **Student** has a *StudentID*, *Name*, *Age*, and *Department*.
- Each **Course** has a *CourseID*, *CourseName*, and *Credits*.
- A Student can enroll in many Courses, and a Course can have many Students.

Practice Question 2

Draw an **Entity–Relationship (ER) diagram** for a **Company Database** with the following requirements:

- The company has **Employees**, **Departments**, and **Projects**.
- Each **Employee** has *EmpID*, *EmpName*, *Salary*, and *Designation*.
- Each **Department** has *DeptID*, *DeptName*, and *Location*.
- Each **Project** has *ProjectID*, *ProjectName*, and *Budget*.
- An Employee works in exactly one Department, but a Department can have many Employees.
- An Employee can work on many Projects, and a Project can have many Employees.